

ONDŘEJ LENGÁL

ZÁKLADY LOGIKY PRO INFORMATIKY—ČÁST 1

FIT VUT V BRNĚ

Výroková logika

1.1 Syntaxe

Před tím, než se pustíme do popisu významu logických formulí, je potřeba určit, jak vůbec logické formule vypadají. Jinými slovy, je potřeba nadefinovat jejich *syntaxi*¹. S logickými formulemi se čtenář jistě již setkal, takže intuitivně asi již tuší, že zatímco řetězce

- „ $(x_1 \wedge x_2) \rightarrow x_3$ “ nebo
- „ $(x \wedge y \wedge \neg x) \vee (\neg x \wedge \neg y \wedge z)$ “

značí logické formule, tak řetězce

- „ $x \rightarrow$ “,
- „ $\wedge \vee y$ “ nebo
- „ $x \wedge y \vee z$ “²

logické formule *ne*značí.

Syntaxe určuje, jak správně zapsat *formule* výrokové logiky. Podobně jako u programovacích jazyků se i u výrokové logiky syntaxe definuje pomocí

1. *abecedy*, tj., množiny symbolů, které se ve formulích mohou vyskytovat a
2. *gramatiky*, tj., pravidel, pomocí nichž můžeme pomocí symbolů z abecedy stavět formule.

Nejdříve si definujme, jak vypadá abeceda výrokové logiky. Uvažujme spočetně nekonečnou množinu³ *výrokových proměnných* $\mathbb{X} = \{x, y, z, \dots, x_1, x_2, \dots\}$. *Abeceda výrokové logiky* je množina $\mathbb{X} \cup \{0, 1, \neg, \wedge, \vee, \rightarrow, \leftrightarrow, (,)\}$, kde symbolům $\{\neg, \wedge, \vee, \rightarrow, \leftrightarrow\}$ říkáme *logické spojky* a symbolům „0“ a „1“ říkáme *logické konstanty*. *Formule výrokové logiky* (v této kapitole budeme používat krátce termín *formule*)

¹ Někdy se také můžete setkat s pojmem *(formální) jazyk*

² Není-li definována priorita operátorů $\wedge$ a $\vee$, není jasné, zda má zápis být interpretován jako formule „ $(x \wedge y) \vee z$ “ nebo jako formule „ $x \wedge (y \vee z)$ “. Tyto dvě formule mají různou sémantiku.

³ Množina je *spočetně nekonečná* pokud je nekonečná a pokud její prvky lze jednoznačně očíslovat přirozenými čísly. Příklady spočetně nekonečných množin jsou například množiny přirozených, celých a racionálních čísel $\mathbb{N}$, $\mathbb{Z}$ a $\mathbb{Q}$. Příkladem nespočetně nekonečné množiny je např. množina reálných čísel $\mathbb{R}$. O spočetných a nespočetných množinách se dozvíte více v libovolném textu o teorii množin.

jsou pak řetězce symbolů, které můžeme nad touto abecedou tvořit pomocí následujících pravidel:

1. Je-li x výroková proměnná, tj. $x \in \mathbb{X}$, pak řetězce „ x “, „0“ a „1“ jsou formule.
2. Jsou-li φ a ψ formule, pak jsou formule i řetězce „ $(\neg\varphi)$ “, „ $(\varphi \wedge \psi)$ “, „ $(\varphi \vee \psi)$ “, „ $(\varphi \rightarrow \psi)$ “ a „ $(\varphi \leftrightarrow \psi)$ “.
3. Formule výrokové logiky jsou právě všechny konečné řetězce získané pomocí předchozích dvou pravidel.

Pro množinu všech formulí výrokové logiky vytvořených pomocí výše uvedených pravidel bude používat označení Φ_{VL} .

Poznámka 1.1

Předchozí definice gramatiky formulí výrokové logiky lze také nahradit zápisem ve stylu Backus-Naurovy formy (BNF) [1],[4] známé z definice syntaxe např. programovacích jazyků nebo datových formátů:

$$\varphi ::= x \in \mathbb{X} \mid 0 \mid 1 \mid (\neg\varphi) \mid (\varphi \wedge \varphi) \mid (\varphi \vee \varphi) \mid (\varphi \rightarrow \varphi) \mid (\varphi \leftrightarrow \varphi)$$

Příklad 1.1

Následující řetězce jsou příklady formulí výrokové logiky:

- „ $((x \wedge y) \vee z)$ “,
- „ $(x_{10} \rightarrow (1 \wedge (\neg z)))$ “ a
- „ $((z \wedge y) \leftrightarrow (z \vee y))$ “.

Následující řetězce *nejsou* formule výrokové logiky:

- „ $x \rightarrow$ “,
- „ $\wedge \vee y$ “ a
- „ $x \wedge y \vee z$ “.

Formule sestavené striktně pomocí výše uvedené gramatiky trpí nepříjemným nadbytkem závorek. Jako příklad uveďme formule

$$(x_1 \wedge (x_2 \wedge (x_3 \wedge x_4))) \text{ a } (((x_1 \wedge x_2) \wedge x_3) \wedge x_4).$$

Jde o dvě různé formule, která ale mají (jak si ukážeme v Sekci 1.2) stejnou sémantiku. Z toho důvodu zavedeme následující konvenci, která určuje, kdy není potřeba závorky psát:

- kolem celé formule:

$$„(x \wedge (y \rightarrow z))“ \rightsquigarrow „x \wedge (y \rightarrow z)“,$$

- u asociativních spojek⁴ ($\wedge, \vee, \leftrightarrow$):

$$„(x \wedge (y \wedge z)) \rightarrow w“ \rightsquigarrow „(x \wedge y \wedge z) \rightarrow w“,$$

⁴ viz Sekce 1.2

- kolem negace:

$$\neg(x \wedge y) \vee ((\neg x) \leftrightarrow z) \rightsquigarrow \neg(x \wedge y) \vee (\neg x \leftrightarrow z)$$

Příklad 1.2

Formule z Příkladu 1.1 můžeme pomocí výše uvedené konvence zapsat následujícím způsobem:

- „ $((x \wedge y) \vee z)$ “ $\rightsquigarrow$ „ $(x \wedge y) \vee z$ “,
- „ $(x_{10} \rightarrow (1 \wedge (\neg z)))$ “ $\rightsquigarrow$ „ $(x_{10} \rightarrow (1 \wedge \neg z))$ “ a
- „ $((z \wedge y) \leftrightarrow (z \vee y))$ “ $\rightsquigarrow$ „ $(z \wedge y) \leftrightarrow (z \vee y)$ “.

Poznámka 1.2

Někdy se můžete setkat s dalšími pravidly pro vynechávání závorek, například když je dána tzv. *precedence* logických spojek, která určuje, jak „těsně“ dané logické spojky vážou své operandy. Precedenci operátorů znáte např. z aritmetiky, kde násobení má vyšší precedenci než sčítání (a lze tedy místo „ $x + (y \cdot z)$ “ psát „ $x + y \cdot z$ “) nebo programovacích jazyků, kde je precedence operátorů definována ve standardu jazyka (např. jazyk C++20 má definovaných 17 úrovní precedence). V tomto textu si vystačíme s konvencí definovanou výše.

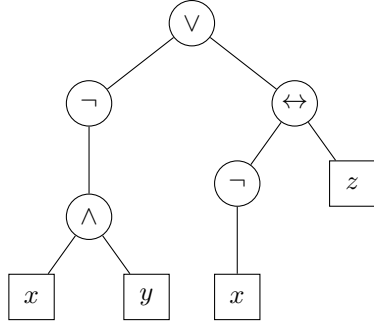
Způsob, jakým je výše definována množina formulí, se nazývá *induktivní* (někdy též *rekurzivní*) definice. Induktivní definice množiny je zadána pomocí

1. *základních prvků* (v našem případě logických konstant 0, 1 a všech výrokových proměnných $x \in \mathbb{X}$) a
2. *konstrukčních pravidel*, které říkají, jak lze za použití prvků z množiny vytvořit prvky nové (v našem případě jak lze za použití logických spojek vytvořit z existujících formulí novou formuli).

Každý prvek z induktivně definované množiny lze reprezentovat jako tzv. (*abstraktní*) *syntaktický strom* vytvořený pomocí základních prvků a konstrukčních pravidel.

Příklad 1.3

Formuli „ $\neg(x \wedge y) \vee ((\neg x) \leftrightarrow z)$ “ lze reprezentovat následujícím syntaktickým stromem:



1.2 Sémantika

Sémantika formule určuje její význam, ve výrokové logice tedy to, kdy *platí*. Například formule $(x \wedge y) \vee z$ platí pokud buď (i) proměnné x a y mají hodnotu 1 nebo (ii) proměnná z má hodnotu 1. Pro definici sémantiky je potřeba nejdříve definovat pojem ohodnocení proměnných. *Ohodnocení proměnných* I je zobrazení, které každé proměnné z $\mathbb{X}$ ⁵ přiřadí hodnotu 0 nebo 1. Formálně:

$$I: \mathbb{X} \rightarrow \{0, 1\} \quad (1.1)$$

Sémantika formule pak určuje, jakou pravdivostní hodnotu formule nabude pro jednotlivá ohodnocení proměnných. Tato hodnota se často definuje induktivně tak, že pro atomickou formuli x odpovídá hodnotě $I(x)$ a pro složenou formuli je definována pomocí tzv. *pravdivostní tabulky*. Pravdivostní tabulka pro dvě formule φ a ψ říká, jaká bude výsledná hodnota formule získané aplikací dané logické spojky na φ a ψ . Pro základní logické spojky vypadá pravdivostní tabulka následovně:

φ	ψ	$\neg\varphi$	$\varphi \vee \psi$	$\varphi \wedge \psi$	$\varphi \rightarrow \psi$	$\varphi \leftrightarrow \psi$
0	0	1	0	0	1	1
0	1	1	1	0	1	0
1	0	0	1	0	0	0
1	1	0	1	1	1	1

Příklad 1.4

1. Pravdivostní tabulka pro formuli $x \rightarrow (x \wedge \neg y)$ bude vypadat následovně:

x	y	$\neg y$	$x \wedge \neg y$	$x \rightarrow (x \wedge \neg y)$
0	0	1	0	1
0	1	0	0	1
1	0	1	1	1
1	1	0	0	0

⁵ případně jen *každé* proměnné v *uvažované* formuli

Tabulku jsme sestrojili tak, že v levé části jsme vypsali všechny možnosti jak proměnným x a y vyskytujícím se ve formuli přiřadit pravdivostní hodnotu a v pravé části jsme si vypsali všechny podformule dané formule a pro každou spočítali její hodnotu pro všechna možná ohodnocení x a y . Poslední (zvýrazněný) sloupec pak udává, jaká bude hodnota formule pro jednotlivá ohodnocení proměnných (v tomto případě to bude vždy 1 kromě případu, kdy x a y mají obě hodnotu 1).

2. Pravdivostní tabulka pro formuli $\varphi: (x \wedge (y \rightarrow z)) \leftrightarrow (x \vee \neg z)$ bude vypadat následovně:

x	y	z	$y \rightarrow z$	$x \wedge (y \rightarrow z)$	$\neg z$	$x \vee \neg z$	φ
0	0	0	1	0	1	1	0
0	0	1	1	0	0	0	1
0	1	0	0	0	1	1	0
0	1	1	1	0	0	0	1
1	0	0	1	1	1	1	1
1	0	1	1	1	0	1	1
1	1	0	0	0	1	1	0
1	1	1	1	1	0	1	1

Poznámka 1.3

Formálně bychom mohli definovat sémantiku formule následujícím způsobem. Nechť $\mathbb{V}$ je množina všech ohodnocení proměnných, tj. $\mathbb{V} = (\mathbb{X} \rightarrow \{0, 1\})$. Potom sémantika výrokové formule je funkce $\llbracket \cdot \rrbracket : \mathbb{V} \rightarrow \{0, 1\}$, např. $\llbracket \varphi \rrbracket$, definována induktivně pomocí pravdivostní tabulky.

Je zřejmé, že pravdivostní tabulka pro formuli s n různými proměnnými bude mít 2^n řádků.

Cvičení 1.1

- Kolik existuje formulí s dvěmi proměnnými?
- Kolik existuje formulí s proměnnými z množiny $\mathbb{X}$?
- Kolik existuje formulí s dvěmi proměnnými s *různou sémantikou*?
Neboli, kolik existuje různých binárních logických spojek?
- Kolik existuje formulí s n proměnnými s *různou sémantikou*?

Věta 1.1. *Logické spojky $\wedge, \vee, \leftrightarrow$ jsou asociativní. Logická spojka $\rightarrow$ asociativní není.*

Důkaz. Nejprve pomocí pravdivostní tabulky dokážeme asociativitu spojek $\wedge$ a $\vee$.

φ	ψ	χ	$\varphi \wedge \psi$	$(\varphi \wedge \psi) \wedge \chi$	$\psi \wedge \chi$	$\varphi \wedge (\psi \wedge \chi)$	$\varphi \vee \psi$	$(\varphi \vee \psi) \vee \chi$	$\psi \vee \chi$	$\varphi \vee (\psi \vee \chi)$
0	0	0	0	0	0	0	0	0	0	0
0	0	1	0	0	0	0	0	1	1	1
0	1	0	0	0	0	0	1	1	1	1
0	1	1	0	0	1	0	1	1	1	1
1	0	0	0	0	0	0	1	1	0	1
1	0	1	0	0	0	0	1	1	1	1
1	1	0	1	0	0	0	1	1	1	1
1	1	1	1	1	1	1	1	1	1	1

Asociativita spojky $\wedge$ plyne z toho, že sloupec pro formuli $(\varphi \wedge \psi) \wedge \chi$ je stejný jako sloupec pro formuli $\varphi \wedge (\psi \wedge \chi)$ (oba sloupce jsou v tabulce zvýrazněny). Podobně pak pro spojku $\vee$ (v levé části tabulky).

Dále stejným způsobem dokážeme asociativitu spojky $\leftrightarrow$ a neasociativitu spojky $\rightarrow$.

φ	ψ	χ	$\varphi \leftrightarrow \psi$	$(\varphi \leftrightarrow \psi) \leftrightarrow \chi$	$\psi \leftrightarrow \chi$	$\varphi \leftrightarrow (\psi \leftrightarrow \chi)$	$\varphi \rightarrow \psi$	$(\varphi \rightarrow \psi) \rightarrow \chi$	$\psi \rightarrow \chi$	$\varphi \rightarrow (\psi \rightarrow \chi)$
0	0	0	1	0	1	0	1	0	1	1
0	0	1	1	1	0	1	1	1	1	1
0	1	0	0	1	0	1	1	0	0	1
0	1	1	0	0	1	0	1	1	1	1
1	0	0	0	1	1	1	0	1	1	1
1	0	1	0	0	0	0	0	1	1	1
1	1	0	1	0	0	0	1	0	0	0
1	1	1	1	1	1	1	1	1	1	1

V pravdivostní tabulce výše lze vidět, že spojka $\leftrightarrow$ skutečně je asociativní. Oproti tomu, spojka $\rightarrow$ asociativní *není* (v pravé části tabulky jsou červeně vyznačeny řádky, kde se hodnota $(\varphi \rightarrow \psi) \rightarrow \chi$ a $\varphi \rightarrow (\psi \rightarrow \chi)$ liší. $\square$

1.3 Terminologie

Nyní zavedeme pojmy které se často používají když mluvíme o formulích výrokové logiky. Ohodnocení proměnných $I: \mathbb{X} \rightarrow \{0, 1\}$ *splňuje* formuli φ tehdy, když platí, že po dosazení hodnot proměnných v ohodnocení I do formule bude výsledná pravdivostní hodnota formule 1. Jinými slovy, pokud sestrojíme pravdivostní tabulku pro φ a na řádku odpovídajícímu ohodnocení proměnných I bude v posledním sloupci tabulky hodnota 1. V takovém případě říkáme, že I je *modelem* formule φ , což značíme jako $I \models \varphi$ („ I splňuje φ “). Opačnou vlastnost („ I nesplňuje φ “) značíme

$I \not\models \varphi$. Pro to, abychom zjistili, zda I je či není modelem φ , ovšem není potřeba sestavit celou pravdivostní tabulku pro φ , ale stačí ji vytvořit jen pro řádek odpovídající I .

Příklad 1.5

Určíme, zda ohodnocení proměnných $I_1 = \{x \mapsto 1, y \mapsto 0, z \mapsto 1\}$ splňuje formuli $\varphi: (x \wedge (y \rightarrow z)) \leftrightarrow (x \vee \neg z)$. Budeme postupovat tak, že si sestojíme řádek pro I_1 z pravdivostní tabulky pro φ :

x	y	z	$y \rightarrow z$	$x \wedge (y \rightarrow z)$	$\neg z$	$x \vee \neg z$	φ
1	0	1	1	1	0	1	1

Z tabulky plyne, že I_1 je modelem formule φ , tedy $I_1 \models \varphi$.

Existuje-li nějaké ohodnocení proměnných I takové, že $I \models \varphi$, pak říkáme, že formule φ je *splnitelná*. Z pravdivostní tabulky poznáme splnitelnou formuli jednoduše tak, že se v posledním sloupci vyskytuje alespoň jednou hodnota 1. Formule φ je *nesplnitelná* (neboli *kontradikce*), pokud není splnitelná, tj., v posledním sloupci pravdivostní tabulky pro φ jsou samé hodnoty 0. Formule φ je *platná* (neboli *tautologie*) pokud je splněna libovolným ohodnocením proměnných, což zapisujeme jako $\models \varphi$. Pomocí pravdivostní tabulky můžeme platnou formuli poznat tak, že v posledním sloupci tabulky jsou samé hodnoty 1. φ je *neplatná* (značeno $\not\models \varphi$) pokud existuje ohodnocení proměnných, které je nesplňuje. Pomocí pravdivostní tabulky bychom takovou formuli poznali tak, že by v posledním sloupci byla alespoň jedna hodnota 0. Tyto pojmy lze rozšířit na *množiny* formulí tak, že množina formulí je *platná* (resp. *splnitelná*) pokud je konjunkce všech formulí v množině *platná* (resp. *splnitelná*).

Příklad 1.6

Zkoumejme typy následujících tří formulí:

$$\begin{aligned}\varphi_1: & (x \wedge y) \rightarrow x \\ \varphi_2: & (x \vee y) \rightarrow x \\ \varphi_3: & (x \vee y) \wedge \neg(x \vee y)\end{aligned}$$

Jejich pravdivostní tabulky vypadají následovně:

x	y	$x \wedge y$	$(x \wedge y) \rightarrow x$	$x \vee y$	$(x \vee y) \rightarrow x$	$\neg(x \vee y)$	$(x \vee y) \wedge \neg(x \vee y)$
0	0	0	1	0	1	1	0
0	1	0	1	1	0	0	0
1	0	0	1	1	1	0	0
1	1	1	1	1	1	0	0

Z tabulek lze vidět následující:

- φ_1 je *platná* i *splnitelná*
- φ_2 je *neplatná*, ale *splnitelná*
- φ_3 je *neplatná* a *nesplnitelná*

Z předchozího příkladu lze vidět, že množinu všech formulí výrokové logiky Φ_{VL} lze rozdělit do tří tříd:

1. tautologie,
2. splnitelné, ale neplatné formule a
3. kontradikce.

V tomto místě je vhodné poznamenat, že $\not\models \varphi$ není to samé jako $\models \neg\varphi$. Vizte následující příklad:

Příklad 1.7

Uvažujme formuli $\varphi: (x \vee y) \rightarrow x$. Její pravdivostní tabulka a pravdivostní tabulka pro její negaci je uvedena níže:

x	y	$x \vee y$	$(x \vee y) \rightarrow x$	$\neg((x \vee y) \rightarrow x)$
0	0	0	1	0
0	1	1	0	1
1	0	1	1	0
1	1	1	1	0

Z tabulky můžeme vidět, že φ není platná (tj. $\not\models \varphi$), ale ani $\neg\varphi$ není platná (tj. $\not\models \neg\varphi$).

Platí ale následující duality.

Věta 1.2. *Platí následující:*

1. *Formule je platná právě tehdy, když její negace je nesplnitelná.*
2. *Formule je splnitelná právě tehdy, když její negace je neplatná.*

Důkaz. Lze dokázat jednoduše analýzou pravdivostních tabulek. $\square$

Dvě formule φ a ψ jsou (*logicky*) *ekvivalentní*, zapisováno $\varphi \Leftrightarrow \psi$, pokud pro všechna ohodnocení proměnných I platí, že I je modelem φ právě tehdy, když I je modelem ψ . Z pravdivostní tabulky bychom logicky ekvivalentní formule poznali tak, že by na všech řádcích tabulky měly stejné hodnoty.

Příklad 1.8

Ukážeme, že formule $\varphi: (x \wedge y) \rightarrow z$ a formule $\psi: \neg x \vee \neg y \vee z$ jsou logicky ekvivalentní.

x	y	z	$x \wedge y$	$(x \wedge y) \rightarrow z$	$\neg x$	$\neg y$	$\neg x \vee \neg y \vee z$
0	0	0	0	1	1	1	1
0	0	1	0	1	1	1	1
0	1	0	0	1	1	0	1
0	1	1	0	1	1	0	1
1	0	0	0	1	0	1	1
1	0	1	0	1	0	1	1
1	1	0	1	0	0	0	0
1	1	1	1	1	0	0	1

Jak vidno, sloupce pro φ a ψ jsou stejné.

Formule ψ je *logickým důsledkem* formule φ , zapisováno $\varphi \Rightarrow \psi$, tehdy, když pro každé ohodnocení proměnných I platí, že je-li I modelem φ , pak je I rovněž modelem ψ .

1.4 Algebraické úpravy formulí

Často je vhodné s formulemi výrokové logiky různým způsobem manipulovat a upravovat je při zachování jejich sémantiky (např. při *minimalizaci* formulí pro účely kompaktnější hardwarové reprezentace nebo při převodu do některé z normálních forem, viz Sekce 1.5). K tomuto slouží *algebraické úpravy*, které nám umožňují s formulemi pracovat jako s výrazy. Pro algebraické úpravy nám slouží logické ekvivalence v následující Větě. Tyto ekvivalence říkají, jakým způsobem můžeme přepsat podformule ve formuli tak, aby byla zachována sémantika.

Věta 1.3. *Nechť x , y a z jsou výrokové formule. Pak platí následující logické ekvivalence:*

$x \wedge (y \wedge z)$	$\Leftrightarrow$	$(x \wedge y) \wedge z$	(asociativita)
$x \vee (y \vee z)$	$\Leftrightarrow$	$(x \vee y) \vee z$	(asociativita)
$x \wedge y$	$\Leftrightarrow$	$y \wedge x$	(komutativita)
$x \vee y$	$\Leftrightarrow$	$y \vee x$	(komutativita)
$x \wedge (y \vee z)$	$\Leftrightarrow$	$(x \wedge y) \vee (x \wedge z)$	(distributivita)
$x \vee (y \wedge z)$	$\Leftrightarrow$	$(x \vee y) \wedge (x \vee z)$	(distributivita)
$\neg(x \wedge y)$	$\Leftrightarrow$	$\neg x \vee \neg y$	(De Morganův zákon)
$\neg(x \vee y)$	$\Leftrightarrow$	$\neg x \wedge \neg y$	(De Morganův zákon)
$x \wedge (x \vee y)$	$\Leftrightarrow$	x	(absorpce)
$x \vee (x \wedge y)$	$\Leftrightarrow$	x	(absorpce)
x	$\Leftrightarrow$	$\neg\neg x$	(dvojitá negace)

$x \vee x$	$\Leftrightarrow$	x	$x \wedge x$	$\Leftrightarrow$	x	(<i>idempotence</i>)
$x \vee 0$	$\Leftrightarrow$	x	$x \wedge 1$	$\Leftrightarrow$	x	(<i>neutralita</i>)
$x \vee 1$	$\Leftrightarrow$	1	$x \wedge 0$	$\Leftrightarrow$	0	(<i>anihilace</i>)
$x \wedge \neg x$	$\Leftrightarrow$	0	$x \vee \neg x$	$\Leftrightarrow$	1	(<i>komplementarita</i>)
$x \rightarrow y$	$\Leftrightarrow$	$\neg x \vee y$				(<i>implikace</i>)
$x \rightarrow y$	$\Leftrightarrow$	$\neg(x \wedge \neg y)$				(<i>implikace</i>)
$x \leftrightarrow y$	$\Leftrightarrow$	$(x \rightarrow y) \wedge (y \rightarrow x)$				(<i>bikondicionál</i>)

Důkaz. Každá z ekvivalencí lze dokázat jednoduše například pomocí pravdivostní tabulky. $\square$

Příklad 1.9

Pomocí algebraických úprav převedeme formuli $(x \wedge \neg y) \vee \neg z$ do tvaru, ve kterém se vyskytují pouze spojky $\neg$ a $\rightarrow$:

$$\begin{aligned}
 (x \wedge \neg y) \vee \neg z &\Leftrightarrow \neg \neg (x \wedge \neg y) \vee \neg z && \{ \text{dvojitá negace} \} \\
 &\Leftrightarrow \neg (x \rightarrow y) \vee \neg z && \{ \text{implikace} \} \\
 &\Leftrightarrow (x \rightarrow y) \rightarrow \neg z && \{ \text{implikace} \}
 \end{aligned}$$

Všimněte si, že v prvním kroku jsme použili pravidlo pro dvojitou negaci *zprava doleva*. Díky použití tohoto pravidla jsme pak mohli použít pravidlo pro implikaci.

Další příklady použití algebraických úprav uvidíme v Sekci 1.5.

1.5 Normální formy

Při mnoha aplikacích výrokové logiky v informatice pracujeme s formulami v některé z *normálních forem*, tj. s formulami která splňují jistá syntaktická omezení. V této sekci si představíme tři takové normální formy.

1.5.1 Negační normální forma (NNF)

Negační normální forma (NNF) je ze zde obsažených normálních forem nejobecnější. Formule je v NNF pokud:

1. obsahuje jen následující logické spojky: $0, 1, \neg, \wedge, \vee$ a
2. negace $\neg$ se vyskytuje jen před proměnnými.

Často se setkáme s pojmem *literál*, který slouží pro označení proměnné nebo její negace. Např., obě následující formule jsou literály:

1. X ,
2. $\neg Y$.

Formule v NNF si pak lze představit jako literály spojené konjunkcemi a disjunkcemi.

Příklad 1.10

Následující formule *jsou* v NNF:

1. $x \wedge \neg y$,
2. $(x \vee y) \wedge (\neg z \vee (\neg x \wedge \neg y))$,
3. x .

Následující formule *nejdou* v NNF:

1. $\neg(x \wedge y)$ (negace je před konjunkcí),
2. $x \wedge \neg\neg y$ (negace je před negací),
3. $x \rightarrow \neg y$ (obsahuje spojku $\rightarrow$).

Libovolná formule lze převést do NNF následujícím algoritmem:

1. Přepíšeme postupně všechny bikondicionály $\leftrightarrow$ ve formuli za implikace pomocí pravidla (*bikondicionál*) z Věty 1.3:

$$x \leftrightarrow y \quad \rightsquigarrow \quad (x \rightarrow y) \wedge (y \rightarrow x)$$

2. Přepíšeme postupně všechny implikace $\rightarrow$ ve formuli za negaci a disjunkci pomocí prvního pravidla (*implikace*) z Věty 1.3:

$$x \rightarrow y \quad \rightsquigarrow \quad \neg x \vee y$$

3. Pomocí De Morganových zákonů postupně přesuneme negaci co nejhlouběji:

$$\neg(x \wedge y) \quad \rightsquigarrow \quad \neg x \vee \neg y$$

$$\neg(x \vee y) \quad \rightsquigarrow \quad \neg x \wedge \neg y$$

4. Kdykoliv to jde, eliminujeme dvojistou negaci:

$$\neg\neg x \quad \rightsquigarrow \quad x$$

Příklad 1.11

Ukážeme si, jak převést formuli $\neg(x \rightarrow \neg(x \wedge y))$ do NNF:

$$\begin{aligned} \neg(x \rightarrow \neg(x \wedge y)) &\Leftrightarrow \neg(\neg x \vee \neg(x \wedge y)) && \text{[implikace]} \\ &\Leftrightarrow x \wedge (x \wedge y) && \text{[De Morgan]} \\ &\Leftrightarrow x \wedge y && \text{[asoc. \& idem.]} \end{aligned}$$

Podotkněme, že poslední krok není nezbytný, jelikož formule $x \wedge (x \wedge y)$ již je v NNF (byť lze ještě zjednodušit).

1.5.2 Disjunktivní normální forma (DNF)

Formule je v *disjunktivní normální formě* (DNF)⁶, pokud má následující tvar:

$$\bigvee_i \bigwedge_j \ell_{i,j} \quad (1.2)$$

kde $\ell_{i,j}$ je literál (tedy buď proměnná, nebo negace proměnné), tedy jde o disjunkci konjunkcí literálů. V případě DNF se konjunkci literálů říká *klauzule* (např. $(x \wedge y \wedge \neg z)$ je klauzule). Následují příklady formulí v DNF:

1. $(x \wedge y \wedge \neg z) \vee (\neg x \wedge \neg y \wedge z) \vee (x \wedge \neg y \wedge z)$,
2. $(x_1 \wedge x_2 \wedge x_3 \wedge x_4 \wedge \neg x_5) \vee (x_2 \wedge \neg x_3 \wedge x_5)$,
3. $(x \wedge y) \vee \neg z$,
4. $x \wedge \neg y$,
5. $x \vee \neg y$,
6. 1.
7. 0.

U prvních dvou formulí je podmínka DNF jasná. U Formule 3 obsahuje druhá klauzule ($\neg z$) jen jeden literál, lze si tedy představit, že to je jednoprvková konjunkce. Formuli 3 by šlo tedy zapsat jako $(x \wedge y) \vee (\neg z)$. Disjunkce ve Formuli 4 obsahuje jen jednu klauzuli $(x \wedge \neg y)$. Oproti tomu disjunkce ve Formuli 5 obsahuje dvě jednoduché klauzule (x) a $(\neg y)$. Formule by teda šla zapsat jako $(x) \wedge (\neg y)$. Formule 6 je disjunkce obsahující jednu prázdnou konjunkci (neutrální prvek pro konjunkci je 1). Oproti tomu Formule 7 je prázdná disjunkce (neutrální prvek pro disjunkci je 0).

1.5.3 Konjunktivní normální forma (CNF)

Formule je v *konjunktivní normální formě* (CNF)⁷, pokud má následující tvar:

$$\bigwedge_i \bigvee_j \ell_{i,j} \quad (1.3)$$

⁶ někdy se také používá označení *sum of products* (SoP)

⁷ někdy se také používá označení *product of sums* (PoS)

kde $\ell_{i,j}$ je literál (tedy buď proměnná, nebo negace proměnné), tedy jde o konjunkci disjunkcí literálů. V případě CNF se disjunkcí literálů říká *klauzule* (např. $(x \vee y \vee \neg z)$ je klauzule)⁸. Následují příklady formulí v CNF:

1. $(x \vee y \vee \neg z) \wedge (\neg x \vee \neg y \vee z) \wedge (x \vee \neg y \vee z)$,
2. $(x_1 \vee x_2 \vee x_3 \vee x_4 \vee \neg x_5) \wedge (x_2 \vee \neg x_3 \vee x_5)$,
3. $(x \vee y) \wedge \neg z$,
4. $x \vee \neg y$,
5. $x \wedge \neg y$,
6. 0.
7. 1.

⁸ pro odlišení klauzulí v DNF a v CNF se klauzulím ve tvaru $(x \wedge y \wedge \neg z)$ někdy říká *konjunktivní klazule* a klauzulím ve tvaru $(x \vee y \vee \neg z)$ se někdy říká *disjunktivní klazule*

Zdůvodnění podmínky CNF pro příklady výše je obdobné jako u DNF.

1.6 Převody do DNF a CNF pomocí algebraických úprav

První způsob, jak převést formuli do DNF nebo CNF je pomocí algebraických úprav ze Sekce 1.4. Konkrétně lze pro převod formule φ do DNF použít následující algoritmus:

1. Převědeme formuli φ na formuli φ_{NNF} v NNF pomocí algoritmu ze Sekce 1.5.1.
2. Formuli φ_{NNF} převedeme do tvaru, kde jsou všechny konjunkce pod disjunkcemi pomocí následujícího distributivního zákona:

$$x \wedge (y \vee z) \rightsquigarrow (x \wedge y) \vee (x \wedge z)$$

Převod formule φ do CNF probíhá obdobně, jen je použit duální distributivní zákon:

1. Převědeme formuli φ na formuli φ_{NNF} v NNF pomocí algoritmu ze Sekce 1.5.1.
2. Formuli φ_{NNF} převedeme do tvaru, kde jsou všechny disjunkce pod konjunkcemi pomocí následujícího distributivního zákona:

$$x \vee (y \wedge z) \rightsquigarrow (x \vee y) \wedge (x \vee z)$$

Příklad 1.12

Ukážeme si, jak převést formuli $((x \vee z) \wedge \neg \neg y) \vee (z \rightarrow w)$ do DNF a CNF:

1. Převod do DNF:

$$\begin{aligned} ((x \vee z) \wedge \neg \neg y) \vee (z \rightarrow w) &\Leftrightarrow ((x \vee z) \wedge y) \vee (z \rightarrow w) && \text{[dvojitá negace]} \\ &\Leftrightarrow ((x \vee z) \wedge y) \vee (\neg z \vee w) && \text{[implikace (NNF)]} \\ &\Leftrightarrow ((x \wedge y) \vee (z \wedge y)) \vee (\neg z \vee w) && \text{[distributivita]} \\ &\Leftrightarrow (x \wedge y) \vee (z \wedge y) \vee \neg z \vee w && \text{[asociativita (DNF)]} \end{aligned}$$

Formuli jsme nejdříve převedli do NNF a poté jednou použili distributivní zákon. Nakonec jsme smazali zbytečné závorky.

2. Převod do CNF:

$$\begin{aligned}
 ((x \vee z) \wedge \neg \neg y) \vee (z \rightarrow w) &\Leftrightarrow ((x \vee z) \wedge y) \vee (z \rightarrow w) && \text{[dvojité negace]} \\
 &\Leftrightarrow ((x \vee z) \wedge y) \vee (\neg z \vee w) && \text{[implikace (NNF)]} \\
 &\Leftrightarrow ((x \vee z) \vee (\neg z \vee w)) \wedge (y \vee (\neg z \vee w)) && \text{[distributivita]} \\
 &\Leftrightarrow (x \vee z \vee \neg z \vee w) \wedge (y \vee \neg z \vee w) && \text{[asociativita (DNF)]} \\
 &\Leftrightarrow (x \vee 1 \vee w) \wedge (y \vee \neg z \vee w) && \text{[komplementarita]} \\
 &\Leftrightarrow 1 \wedge (y \vee \neg z \vee w) && \text{[anihilace]} \\
 &\Leftrightarrow y \vee \neg z \vee w && \text{[neutralita]}
 \end{aligned}$$

Formuli jsme nejdříve převedli do NNF stejně jako při převodu do DNF. Poté jsme jednou použili distributivní zákon a po smazání závorek obdrželi formuli $(x \vee z \vee \neg z \vee w) \wedge (y \vee \neg z \vee w)$ v CNF. Pokračovali jsme ale ještě dalšími zjednodušovacími úpravami a dospěli k ekvivalentní formuli $y \vee \neg z \vee w$, která je také v CNF (konjunkce má jen jednu klauzuli).

Tady si můžeme povšimnout, že výsledná formule je jak v CNF, tak i v DNF. Proč nám při převodu do DNF vyšla tedy jiná formule? Formule získané algebraickým převodem obecně nejsou *kanonické*, tj. výstupem převodu není jedna unikátní formule. V našem příkladu bychom mohli formuli získanou při převodu do DNF v Bodě 1 ještě dále upravovat:

$$\begin{aligned}
 (x \wedge y) \vee (z \wedge y) \vee \neg z \vee w &\Leftrightarrow (x \wedge y) \vee ((z \vee \neg z) \wedge (y \vee \neg z)) \vee w && \text{[distributivita]} \\
 &\Leftrightarrow (x \wedge y) \vee (1 \wedge (y \vee \neg z)) \vee w && \text{[anihilace]} \\
 &\Leftrightarrow (x \wedge y) \vee (y \vee \neg z) \vee w && \text{[neutralita]} \\
 &\Leftrightarrow (x \wedge y) \vee y \vee \neg z \vee w && \text{[asociativita]} \\
 &\Leftrightarrow y \vee \neg z \vee w && \text{[absorpce]}
 \end{aligned}$$

Nyní jsme již získali stejnou formuli jako při předchozím převodu.

1.7 Převody do DNF a CNF pomocí pravdivostní tabulky

Je často jednodušší namísto algebraických úprav dělat převody do DNF a CNF pomocí pravdivostní tabulky⁹. Nejdříve si ukážeme, jak pomocí pravdivostní tabulky provést převod formule do DNF.

Základní myšlenka převodu do DNF je jednoduchá: v pravdivostní tabulce najdeme právě všechna ohodnocení proměnných $I_1, \dots, I_\ell$ pro které má formule hodnotu 1. Výsledná formule v DNF se pak sestaví

⁹ Toto platí často jen pro formule s malým počtem proměnných. Při velkém počtu proměnných velikost tabulky neúměrně (exponenciálně) roste, zatímco algebraické úpravy mohou stále pracovat s relativně malými formulemi.

jako disjunkce konjunktivních klauzulí, kde každá klauzule odpovídá právě jednomu ohodnocení I_j pro $1 \leq j \leq \ell$ tak, že se sestaví konjunkce literálů odpovídající tomu, jakou I_j přiřazuje proměnným hodnotu (pokud přiřazuje proměnné x hodnotu 1, pak se vezme literál x , pokud přiřazuje proměnné x hodnotu 0, pak se vezme literál $\neg x$).

Formálně lze tuto konstrukci popsat tak, že pro každé I_j se vytvoří konjunktivní klauzule

$$k_j^{DNF} = \left(\bigwedge_{\substack{x \in \mathbb{X} \\ I_j(x)=0}} \neg x \right) \wedge \left(\bigwedge_{\substack{x \in \mathbb{X} \\ I_j(x)=1}} x \right).$$

a výsledná formule φ v DNF pak vznikne disjunkcí všech klauzulí k_j^{DNF} :

$$\varphi_{DNF} = \bigvee_{1 \leq j \leq \ell} k_j^{DNF}.$$

Poznamenejme ještě, že pokud je formule nespílitelná (nemá pro žádné ohodnocení proměnných hodnotu 1), pak po převodu do DNF dostaneme formuli 0.

Převod formule do CNF probíhá duálně: v pravdivostní tabulce najdeme právě všechna ohodnocení proměnných $I_1, \dots, I_\ell$ pro které má formule hodnotu 0. Výsledná formule v CNF se pak sestaví jako konjunkce disjunktivních klauzulí, kde každá klauzule odpovídá právě jednomu ohodnocení I_j pro $1 \leq j \leq \ell$ tak, že se sestaví disjunkce literálů negujících to, jakou I_j přiřazuje proměnným hodnotu (pokud přiřazuje proměnné x hodnotu 1, pak se vezme literál $\neg x$, pokud přiřazuje proměnné x hodnotu 0, pak se vezme literál x).

Formálně lze tuto konstrukci popsat tak, že pro každé I_j se vytvoří disjunktivní klauzule

$$k_j^{CNF} = \left(\bigvee_{\substack{x \in \mathbb{X} \\ I_j(x)=0}} x \right) \vee \left(\bigvee_{\substack{x \in \mathbb{X} \\ I_j(x)=1}} \neg x \right).$$

a výsledná formule φ v CNF pak vznikne konjunkcí všech klauzulí k_j^{CNF} :

$$\varphi_{CNF} = \bigwedge_{1 \leq j \leq \ell} k_j^{CNF}.$$

Pokud je formule platná (má pro všechna ohodnocení proměnných hodnotu 1), pak po převodu do DNF dostaneme formuli 1.

Příklad 1.13

Ukážeme si převod do DNF a CNF na formuli $\varphi: (x \rightarrow y) \leftrightarrow (y \rightarrow z)$.
Nejdříve si pro formuli sestavíme pravdivostní tabulku:

x	y	z	$x \rightarrow y$	$y \rightarrow z$	φ
0	0	0	1	1	1
0	0	1	1	1	1
0	1	0	1	0	0
0	1	1	1	1	1
1	0	0	0	1	0
1	0	1	0	1	0
1	1	0	1	0	0
1	1	1	1	1	1

V tabulce jsme pro předhlednost zeleně vyznačili ohodnocení proměnných, pro které má formule hodnotu 1 a červeně vyznačili ohodnocení proměnných, pro které má formule hodnotu 0.

1. Převod do DNF: Formule je splněna pro následující čtyři ohodnocení proměnných:

- $I_{000} = \{x \mapsto 0, y \mapsto 0, z \mapsto 0\}$,
- $I_{001} = \{x \mapsto 0, y \mapsto 0, z \mapsto 1\}$,
- $I_{011} = \{x \mapsto 0, y \mapsto 1, z \mapsto 1\}$ a
- $I_{111} = \{x \mapsto 1, y \mapsto 1, z \mapsto 1\}$.

Z těchto ohodnocení proměnných sestavíme následující čtyři konjunktivní klauzule:

- $k_{000}^{DNF}: (\neg x \wedge \neg y \wedge \neg z)$,
- $k_{001}^{DNF}: (\neg x \wedge \neg y \wedge z)$,
- $k_{011}^{DNF}: (\neg x \wedge y \wedge z)$ a
- $k_{111}^{DNF}: (x \wedge y \wedge z)$.

Disjunkce těchto klauzulí pak bude následující formule φ_{DNF} :

$$(\neg x \wedge \neg y \wedge \neg z) \vee (\neg x \wedge \neg y \wedge z) \vee (\neg x \wedge y \wedge z) \vee (x \wedge y \wedge z)$$

2. Převod do CNF: Formule není splněna pro následující čtyři ohodnocení proměnných:

- $I_{010} = \{x \mapsto 0, y \mapsto 1, z \mapsto 0\}$,
- $I_{100} = \{x \mapsto 1, y \mapsto 0, z \mapsto 0\}$,
- $I_{101} = \{x \mapsto 1, y \mapsto 0, z \mapsto 1\}$ a
- $I_{110} = \{x \mapsto 1, y \mapsto 1, z \mapsto 0\}$.

Z těchto ohodnocení proměnných sestavíme následující čtyři disjunktivní klauzule:

- $k_{010}^{CNF} : (x \vee \neg y \vee z)$,
- $k_{100}^{CNF} : (\neg x \vee y \vee z)$,
- $k_{101}^{CNF} : (\neg x \vee y \vee \neg z)$ a
- $k_{110}^{CNF} : (\neg x \vee \neg y \vee z)$.

Konjunkce těchto klauzulí pak bude následující formule φ_{CNF} :

$$(x \vee \neg y \vee z) \wedge (\neg x \vee y \vee z) \wedge (\neg x \vee y \vee \neg z) \wedge (\neg x \vee \neg y \vee z)$$

1.8 Systémy logických spojek

Kromě logických spojek $0, 1, \neg, \wedge, \vee, \rightarrow, \leftrightarrow$, se kterými jsme pracovali v předchozích kapitolách, se často můžeme setkat s dalšími spojkami. Nejčastější z nich jsou následující:

- $\oplus$: *exkluzivní disjunkce* (často se jí též říká „xor“ (z anglického „exclusive or“) nebo „nonekvivalence“), kterou lze definovat např. jako $x \oplus y \stackrel{\text{def}}{\iff} \neg(x \leftrightarrow y)$.
- $\downarrow$: *negovaná disjunkce* (často se používá označení „nor“ z anglického „not or“, můžeme se ale také setkat s termínem *Peircova šipka*): $x \downarrow y \stackrel{\text{def}}{\iff} \neg(x \vee y)$.
- $\uparrow$: *negovaná konjunkce* (často se používá označení „nand“ z anglického „not and“, můžeme se ale také setkat s termínem *Shefferův operátor*): $x \uparrow y \stackrel{\text{def}}{\iff} \neg(x \wedge y)$.

Pomocí pravdivostní tabulky můžeme tyto spojky definovat následujícím způsobem:

x	y	$x \oplus y$	$x \downarrow y$	$x \uparrow y$
0	0	0	1	1
0	1	1	0	1
1	0	1	0	1
1	1	0	0	0

Naskytá se otázka, zda jsou všechny doposud zavedené logické spojky nezbytné, nebo jestli si bez některých vystačíme. Definujme si pojem *systém logických spojek* S , což je nějaká podmnožina logických spojek, např. $S = \{\rightarrow, \wedge, 1\}$. Budeme používat označení Φ_S pro množinu všech formulí, které lze vytvořit pomocí spojek z S (a proměnných z $\mathbb{X}$ a závorek). Systém spojek S je *úplný*, pokud pro každou formuli výrokové logiky $\varphi \in \Phi_{VL}$ existuje formule $\varphi_S \in \Phi_S$ taková, že $\varphi \Leftrightarrow \varphi_S$. Uveďme si příklady úplných a neúplných systémů spojek:

- **úplné:** $\{\wedge, \neg\}$, $\{\vee, \neg\}$, $\{\rightarrow, \neg\}$, $\{\uparrow\}$, $\{\downarrow\}$, $\{\oplus, 1\}$, $\{\rightarrow, 0\}$ (a jejich nadmnožiny)
- **neúplné:** $\{\wedge, \vee, \rightarrow, \leftrightarrow, 1\}$ (a libovolná podmnožina)

Příklad 1.14

Dokážeme, že systém spojek $\{\wedge, \neg\}$ je úplný. Toto můžeme dokázat tak, že ukážeme, jak můžeme pomocí spojek $\wedge$ a $\neg$ vyjádřit zbytek spojek:

$$x \vee y \stackrel{\text{def}}{\iff} \neg(\neg x \wedge \neg y)$$

$$0 \stackrel{\text{def}}{\iff} x \wedge \neg x$$

$$1 \stackrel{\text{def}}{\iff} x \vee \neg x$$

$$x \rightarrow y \stackrel{\text{def}}{\iff} \neg x \vee y$$

$$x \leftrightarrow y \stackrel{\text{def}}{\iff} (x \rightarrow y) \wedge (y \rightarrow x)$$

Všimněte si, že pro definice jednotlivých spojek v uvažovaném systému jsme použili logické ekvivalence z Věty 1.3.

Predikátová logika

Oproti *výrokové logice* nám *predikátová logika* poskytuje mnohem bohatší vyjadřovací prostředky, které se používají nejen v matematice pro zapisování vlastností různých struktur, ale stále více v informatice jako jazyk pro specifikaci systémů, práci s databázemi, analýzu a verifikaci softwaru a hardwaru, umělé inteligenci atp. Ve výrokové logice jsme pracovali s formulemi nad výrokovými proměnnými, které reprezentovaly tzv. „prvotní výroky,“ do nichž jsme se již nedívali. V predikátové logice nás obsah těchto výroků zabývat bude.

V tomto textu se zaměříme na *predikátovou logiku 1. řádu*¹ (v angličtině se často označuje jen termínem *first-order logic (FOL)*). Oproti výrokové logice nám predikátová logika umožňuje mluvit o *entitách* z nějakého *univerza* a jejich *vlastnostech* a *vztazích* mezi nimi (např. o tom, že některá přirozená čísla jsou menší než jiná přirozená čísla).

¹ Existuje i predikátová logika 2. řádu, 3. řádu, atd., které mají ještě větší vyjadřovací schopnosti; těmito se zabývat nebudeme. Zájemci mohou najít hezký úvod do těchto logik např. v [3], případně mohou kontaktovat autora tohoto textu.

Příklad 2.1

Než se pustíme do formálního popisu syntaxe a sémantiky predikátové logiky, ukažme si na několika příkladech formuli predikátové logiky, co pomocí ní můžeme vyjádřit.

1. Uvažujme následující sylogismus (*Sylogismus* je druh logického tvrzení určité formy, kde je jeden výrok odvozen z dvou jiných výroků (předpokladů). Sylogismy zavedl Aristotelés ve 4. století před naším letopočtem a sloužily jako základ pro logickou dedukci až do poloviny 19. století, kdy George Boole položil základy symbolické logiky.):

„Všichni muži jsou smrtelní. Sokrates je muž. Tedy Sokrates je smrtelný.“

Toto tvrzení lze v predikátové logice zapsat například takto:

$$(\forall x (man(x) \rightarrow mortal(x)) \wedge man(Socrates)) \rightarrow mortal(Socrates).$$

2. Tvzení

„Existuje nekonečně mnoho prvočísel.“

lze v predikátové logice zapsat například jako

$$\forall x \exists y \left(y > x \wedge \forall z \left((z \neq 1 \wedge z \neq y) \rightarrow \forall w (wz \neq y) \right) \right).$$

3. Tvzení

„Relace R je tranzitivní.“

můžeme v predikátové logice zapsat například následující formulí:

$$\forall x \forall y \forall z \left((R(x, y) \wedge R(y, z)) \rightarrow R(x, z) \right)$$

4. Uvažujme tabulky $R[\text{jmeno}, \text{id}]$ a $S[\text{id}, \text{vek}]$ v SQL databázi.

Následující SQL dotaz

```
select R.jmeno
from R join S on R.id = S.id
where S.vek = 42
```

Lze vyjádřit např. pomocí této formule predikátové logiky:

$$\exists z (R(x, z) \wedge S(z, 42))$$

kde hodnoty volné proměnné x , pro které formule platí, odpovídají řádkům ve výsledku dotazu.

5. Velká Fermatova věta [6] lze zapsat jako

$$\forall n \forall x \forall y \left(n > 2 \rightarrow \forall z (x^n + y^n \neq z^n) \right)$$

2.1 *Syntaxe*

Syntaxi predikátové logiky lze chápat jako rozšíření syntaxe výrokové logiky.

2.1.1 *Abeceda*

Začneme definicí abecedy, která sestává z následujících prvků:

1. *logické spojky*: $\neg, \wedge, \vee, \rightarrow, \leftrightarrow$,
2. *proměnné*: $x, y, z, \dots, x_1, x_2, \dots \in \mathbb{X}$, kde $\mathbb{X}$ je spočetně nekonečná množina proměnných,
3. *kvantifikátory*: $\exists, \forall$,
4. *závorky*: $), ($,
5. *funkční symboly*: $f_1, f_2, \dots \in \mathcal{F}$,
6. *predikátové symboly*: $p_1, p_2, \dots \in \mathcal{P}$ a

7. *predikátový symbol rovnosti: $=_{/2}$.*

Ve výčtu výše se objevily některé nové termíny, které si teď blíže popíšeme. Tučně vyznačené jsou pojmy **funkční symboly** (z množiny $\mathcal{F}$) a **predikátové symboly** (z množiny $\mathcal{P}$). Funkční a predikátové množiny nejsou pevně zafixovány, ale lze je chápat jako „*parametr*“ jazyka, který si volíme podle toho, co chceme v logice vyjádřit. Každý funkční i predikátový symbol má danu tzv. *aritu*, která říká, kolik parametrů (operandů) daný symbol bere (např. symbol f který bychom používali ve výrazech ve tvaru $f(a, 42, \pi)$ má aritu 3). Aritu lze chápat jako funkci $(\mathcal{F} \cup \mathcal{P}) \rightarrow \mathbb{N}$. Má-li např. funkční symbol f aritu n , budeme to značit jako $f_{/n}$.

Uvedme si nějaké příklady používaných funkčních a predikátových symbolů včetně jejich arity:

- *funkční symboly:*

$+_{/2}$: binární funkční symbol používaný např. pro operaci *sčítání* (např. $x + y + 42$),

$\Delta_{/2}$: binární funkční symbol používaný např. pro operaci *symetrického rozdílu množin* (např. $A \Delta B$),

$\sin_{/1}$: unární funkční symbol používaný pro operaci *sinus* (např. $\sin(2 \cdot x)$),

$e_{/0}$: nulární funkční symbol (tj. symbol pro konstantu) používaný pro *Eulerovo číslo* 2.71828... (např. e^x),

strncpy $_{/2}$, **strlen** $_{/1}$: funkční symboly odpovídající deklaracím funkcí v hlavičkovém souboru **string.h** standardní knihovny jazyka C (např. **strncpy**($y, x, 42$)),

- *predikátové symboly:*

$<_{/2}$: binární predikátový symbol používaný např. pro vyjádření vztahu *ostře menší* (např. $x < 5 + y$),

$|_{/2}$: binární predikátový symbol používaný např. pro vyjádření vztahu *dělí* (např. $42|x$),

isnan $_{/1}$: unární predikátový symbol pro odpovídající makro v hlavičkovém souboru **math.h** standardní knihovny jazyka C vracející hodnotu typu **bool**, které se používá pro určení, zda hodnota výrazu typu **float** je nebo není číslo („*not-a-number: NaN*“) (např. **isnan**($\frac{0}{0}$)),

$S_{/3}$: kde S označuje ternární relaci (použití např. pro popis řádku v tabulce S v databázi: $S(\text{"Chewbacca"}, \text{"Wookiee"}, \text{"Kashyyyk"})$).

Signatura jazyka predikátové logiky je dána jako dvojice $\langle \mathcal{F}, \mathcal{P} \rangle$. Signaturu můžeme chápat jako „*parametr*“ jazyka predikátové logiky², až po dodání signatury můžeme začít tvořit samotné formule predikátové

² Pro znalce C++ si lze signaturu představit např. jako parametr šablony třídy—až po její instanciaci dostaneme samotnou třídu.

logiky. Uvědomme si, že prozatím funkční a predikátové symboly nemají žádný význam; ten jim bude přiřazen až později.

Příklad 2.2

Uveďme si nějaké příklady jazyků predikátové logiky a formulí v těchto jazycích:

1. Jazyk teorie uspořádání se signaturou $\langle \mathcal{F} = \emptyset, \mathcal{P} = \{\leq_{/2}\} \rangle$:
 - $\forall x(x \leq x)$
 - $\forall x \forall y((x \leq y \wedge y \leq x) \rightarrow x = y)$
 - $\forall x \forall y \forall z((x \leq y \wedge y \leq z) \rightarrow x \leq z)$
 - $\forall x \forall y(x \leq y \vee y \leq x)$
2. Jazyk teorie grup se signaturou $\langle \mathcal{F} = \{\cdot_{/2}, e_{/0}\}, \mathcal{P} = \emptyset \rangle$:
 - $\forall x(x \cdot e = x \wedge e \cdot x = x)$
 - $\forall x \exists y(x \cdot y = e \wedge y \cdot x = e)$
3. Jazyk teorie množin se signaturou $\langle \mathcal{F} = \emptyset, \mathcal{P} = \{\in_{/2}\} \rangle$:
 - $\forall u(u \in x \rightarrow u \in y)$
 - $\forall x \exists y \forall z(\forall u(u \in z \rightarrow u \in x) \rightarrow z \in y)$
4. Jazyk teorie polí se signaturou $\langle \mathcal{F} = \{read_{/2}, write_{/3}\}, \mathcal{P} = \emptyset \rangle$:
 - $\forall x \forall y(\forall i(read(x, i) = read(y, i)) \rightarrow x = y)$
5. Jazyk elementární (tzv. Peanovy) aritmetiky se signaturou $\langle \mathcal{F} = \{0_{/0}, S_{/1}, +_{/2}, \cdot_{/2}\}, \mathcal{P} = \emptyset \rangle$:
 - $\forall x \forall y \exists z(x + y = z)$
 - $\forall x \forall y(x \cdot S(y) = x \cdot y + x)$
 - $\exists x \forall y(\neg(x = S(y)))$

V některých příkladech výše jsme použili predikátový symbol $=_{/2}$, který nemusí být v signatuře, protože je v *každém* jazyce predikátové logiky (viz Bod 7 definice abecedy).

2.1.2 Gramatika

Při definici gramatiky predikátové logiky začneme definicí *termu*³. Termy nám budou popisovat, jakým způsobem se spočítá nějaká *hodnota*. Například v jazyce s množinou funkčních symbolů $\{+_{/2}, \cdot_{/2}, 3_{/0}\}$ jsou následující zápisy termy:

- „ x “,
- „ 3 “,
- „ $(x \cdot y) + (x \cdot (y \cdot 3))$ “.

Formálně lze *termy* definovat následujícím způsobem:

1. Je-li x proměnná, tj. $x \in \mathbb{X}$, pak řetězec „ x “ je term.
2. Je-li f funkční symbol s aritou n a $t_1, \dots, t_n$ jsou termy, pak i řetězec „ $f(t_1, \dots, t_n)$ “ je term.

³ *Term* je jen jiné označení pro *výraz*, v logice se však téměř výhradně používá slovo *term*.

Často se používá následující zjednodušení notace:

1. pro symboly binárních operátorů se používá infixový zápis (tj. např. místo „ $+(x, 42)$ “ píšeme nám dobře známé „ $x + 42$ “),
2. pro symboly unárních operátorů se používá prefixový nebo postfixový zápis (např. místo „ $-(x)$ “ se píše „ $-x$ “ a místo „ $^{-1}(x)$ “ se píše „ x^{-1} “),
3. u konstantních symbolů (tj. symbolů s aritou 0) se nepíšou závorky (např. místo „ $\pi()$ “ se píše jen „ π “).

Příklad 2.3

Uvažujme následující jazyky a příklady termů v nich:

1. Jazyk teorie grup se signaturou $\langle \mathcal{F} = \{\cdot/2, e/0\}, \mathcal{P} = \emptyset \rangle$. Příklady termů v tomto jazyce jsou následující:
 - „ $(z \cdot (x \cdot e)) \cdot y$ “,
 - „ e “,
 - „ x “ a
 - „ $e \cdot e$ “.
2. Jazyk teorie polí se signaturou $\langle \mathcal{F} = \{read/2, write/3\}, \mathcal{P} = \emptyset \rangle$. Příklady termů v tomto jazyce jsou následující:
 - „ $read(x, y)$ “,
 - „ x “ a
 - „ $read(write(x, y, z), y)$ “.
3. Jazyk elementární (tzv. Peanovy) aritmetiky se signaturou $\langle \mathcal{F} = \{0/0, S/1, +/2, \cdot/2\}, \mathcal{P} = \emptyset \rangle$. Příklady termů v tomto jazyce jsou následující:
 - „ $x + y$ “,
 - „ $x \cdot S(y + S(0 \cdot x))$ “ a
 - „ x “.

Nyní již můžeme definovat *formule* predikátové logiky:

1. Je-li p predikátový symbol s aritou n a $t_1, \dots, t_n$ jsou termy, potom je řetězec „ $p(t_1, \dots, t_n)$ “ formule (toto platí i pro „vestavěný“ binární predikátový symbol $=$). Formulí tohoto tvaru říkáme *atomická formule*.
2. Jsou-li φ a ψ formule, pak jsou formule i řetězce „ $(\neg \varphi)$ “, „ $(\varphi \wedge \psi)$ “, „ $(\varphi \vee \psi)$ “, „ $(\varphi \rightarrow \psi)$ “ a „ $(\varphi \leftrightarrow \psi)$ “.
3. Je-li φ formule a $x \in \mathbb{X}$ proměnná, pak jsou formule i řetězce „ $(\exists x \varphi)$ “ a „ $(\forall x \varphi)$ “.

Poznámka 2.1

Definice gramatiky pro formuli predikátové logiky φ pomocí Backus-Naurovy formy (BNF) by vypadala následujícím způsobem:

$$\begin{aligned} t &::= x \mid f(t_1, \dots, t_n) \\ \varphi &::= p(t_1, \dots, t_m) \mid t_1 = t_2 \mid \\ &\quad (\neg\varphi) \mid (\varphi \wedge \varphi) \mid (\varphi \vee \varphi) \mid (\varphi \rightarrow \varphi) \mid (\varphi \leftrightarrow \varphi) \mid \\ &\quad (\forall x\varphi) \mid (\exists x\varphi) \end{aligned}$$

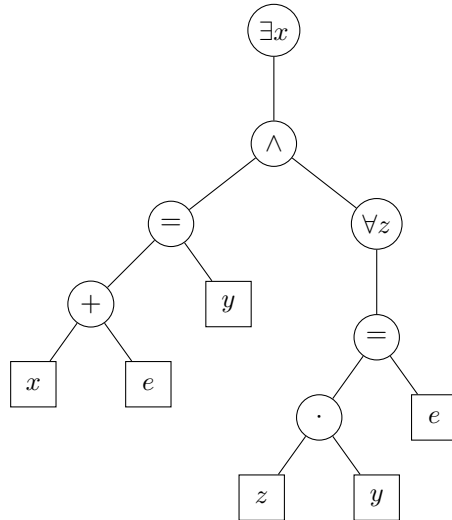
pro $x \in \mathbb{X}$, $f/n \in \mathcal{F}$ a $p/m \in \mathcal{P}$.

Příklady formulí v různých jazycích můžeme vidět v Příkladu 2.2.

Formule predikátové logiky se často (minimálně v programech) reprezentují pomocí jejich syntaktických stromů.

Příklad 2.4

Syntaktický strom formule $\exists x(z + e = y \wedge \forall z(z \cdot y = e))$ může vypadat následujícím způsobem:

**2.1.3 Volné a vázané výskyty proměnných**

Kvantifikátory hrají ve formulích predikátové logiky velkou roli: vážou proměnnou a umožňují nám mluvit o všech prvcích z univerza diskurzu. Jak uvidíme později v Sekci 2.2.1, formule mluví o tom, co musí platit pro proměnné, které nejsou ve formuli kvantifikovány, tzv. volné proměnné.

Uvažme následující formuli:

$$\varphi: p(x) \wedge \forall x \left((\exists y (f(x) = y)) \wedge (p(y) \rightarrow \forall y (x = y)) \right)$$

1. výskyt x
2. výskyt x
3. výskyt x

1. výskyt y
2. výskyt y
3. výskyt y

s vyznačenými výskyty proměnných. Výskyt proměnné z ve formuli je *vázaný*, pokud se nachází v oboru platnosti kvantifikátoru $\exists z$ nebo $\forall z$ (ve formuli $\forall z \psi$ je oborem platnosti kvantifikátoru $\forall z$ formule ψ — v syntaktickém stromě formule by to byl podstrom nacházející se pod uzlem $\forall z$). V uvažované formuli jsou obory platnosti jednotlivých kvantifikátorů vyznačeny v následujícím zápisu:

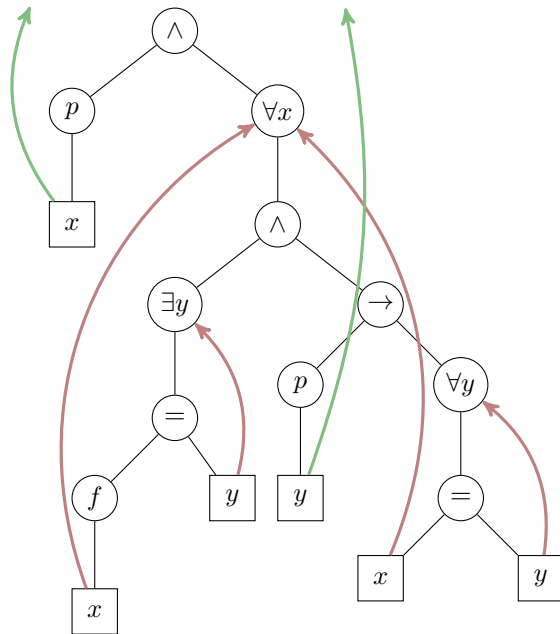
$$p(x) \wedge \forall x \left((\exists y (f(x) = y)) \wedge (p(y) \rightarrow \forall y (x = y)) \right)$$

Pokud je výskyt proměnné vázaný, pak je vázaný nejbližším kvantifikátorem nad sebou. Pokud výskyt proměnné není vázaný žádným kvantifikátorem, pak je *volný*. V následujícím zápise můžeme vidět, kterými kvantifikátory jsou vázány různé výskyty proměnných x a y , a které výskyty jsou volné:

$$p(x) \wedge \forall x \left((\exists y (f(x) = y)) \wedge (p(y) \rightarrow \forall y (x = y)) \right)$$

volný výskyt
volný výskyt

Syntaktický strom formule s vyznačenými volnými a vázanými výskyty proměnných vypadá takto:



Říkáme, že proměnná je ve formuli *volná*, pokud v ní má alespoň jeden volný výskyt. Formuli φ s volnými proměnnými $x_1, \dots, x_n$ často značíme

jako $\varphi(x_1, \dots, x_n)$ a používáme notaci $\text{FREE}[\varphi] \stackrel{\text{def}}{=} \{x_1, \dots, x_n\}$. Formulí s volnými proměnnými říkáme *výroková forma*, formulí bez volných proměnných říkáme *uzavřená formule* nebo také *výrok*.

2.2 Sémantika

Sémantika predikátové logiky je podstatně komplikovanější než u výrokové logiky. Zatímco u výrokové logiky nám stačilo pracovat s ohodnocením proměnných přiřazujícím každé proměnné hodnotu 0 nebo 1, v predikátové logice musíme proměnným přiřazovat hodnoty z nějakého univerza a musíme interpretovat funkční a predikátové symboly. Tomuto zevšeobecnění ohodnocení proměnných se říká *realizace* jazyka (někdy též *interpretace* jazyka). Například, máme-li následující formuli:

$$\forall x(f(y) < x),$$

tak abychom určili, zda tato formule platí, nestačí nám jen znát,

1. jakou hodnotu bude mít proměnná y ,

ale také to,

2. jaké všechny prvky bude uvažovat kvantifikátor „ $\forall x$ “,

3. jaká je sémantika funkčního symbolu „ f_1 “ a

4. jaká je sémantika predikátového symbolu „ $<$ “.

Body 1.–4. nám upřesní právě realizace.

Realizace jazyka predikátové logiky se signaturou $\langle \mathcal{F}, \mathcal{P} \rangle$ je dvojice (D_I, α_I) kde:

- D_I je neprázdná množina zvaná *doména* nebo také *univerzum diskurzu* (zdůrazněme nezbytnou podmínku, že D_I musí být *neprázdná*),
- α_I přiřazuje funkčním a predikátovým symbolům jazyka a proměnným sémantiku následujícím způsobem:

- každému funkčnímu symbolu $f_n \in \mathcal{F}$ přiřazuje n -ární (totální) funkci

$$f_I: \overbrace{D_I \times \dots \times D_I}^n \rightarrow D_I,$$

- každému predikátovému symbolu $p_m \in \mathcal{P}$ přiřazuje m -ární relaci

$$p_I \subseteq \overbrace{D_I \times \dots \times D_I}^m,$$

- každé proměnné $x \in \mathbb{X}$ přiřazuje hodnotu z domény D_I .

Místo $\alpha_I(f)$, $\alpha_I(p)$ a $\alpha_I(x)$ budeme často psát jen $I(f)$, $I(p)$ a $I(x)$. Části I která určuje doménu a interpretaci symbolů z $\mathcal{F}$ a $\mathcal{P}$ se někdy říká *struktura* a části I která určuje hodnoty proměnných $x \in \mathbb{X}$ se někdy říká *ohodnocení proměnných*.

Příklad 2.5

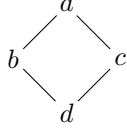
Uvažujme jazyk predikátové logiky se signaturou $\langle \mathcal{F} = \{+_{/2}\}, \mathcal{P} = \emptyset \rangle$. Následují příklady realizací tohoto jazyka:

1. Realizace I_1 modelující sčítání přirozených čísel, kde
 - $D_{I_1} = \mathbb{N}$ a
 - $I_1(+_{/2}) = +_{\mathbb{N}}$ (tj. sčítání přirozených čísel: $+_{\mathbb{N}} = \{(0, 0) \mapsto 0, (0, 1) \mapsto 1, (1, 0) \mapsto 1, (0, 2) \mapsto 2, \dots\}$).
2. Realizace I_2 modelující sčítání tříprvkových vektorů reálných čísel, kde
 - $D_{I_2} = \mathbb{R}^3$ a
 - $I_2(+_{/2}) = \{([x_1, y_1, z_1], [x_2, y_2, z_2]) \mapsto [x_1 +_{\mathbb{R}} x_2, y_1 +_{\mathbb{R}} y_2, z_1 +_{\mathbb{R}} z_2] \mid x_1, x_2, y_1, y_2, z_1, z_2 \in \mathbb{R}\}$, kde $+_{\mathbb{R}}$ je sčítání reálných čísel.
3. Realizace I_3 modelující spojení (supremum) v Booleově algebře nad $\{0, 1\}$.
 - $D_{I_3} = \{0, 1\}$ a
 - $I_3(+_{/2}) = \{(0, 0) \mapsto 0, (0, 1) \mapsto 1, (1, 0) \mapsto 1, (1, 1) \mapsto 1\}$.
4. Realizace I_4 modelující konkatenci řetězců v jazyce Python, kde
 - $D_{I_4} = \text{str}$ (datový typ pro řetězec v jazyce Python) a
 - $I_4(+_{/2}) = \{("ab", "cd") \mapsto "abcd", ("foo", "bar") \mapsto "foobar", \dots\}$.

Pozn. v realizacích výše neuvádíme ohodnocení proměnných; to dává smysl uvádět až v případě, kdy mluvíme o formuli s volnými proměnnými.

Dále uvažujme jazyk predikátové logiky se signaturou $\langle \mathcal{F} = \{+_{/2}, \cdot_{/2}, -_{/1}\}, \mathcal{P} = \{E_{/1}\} \rangle$. Následují příklady realizací tohoto jazyka:

5. Realizace I_5 modelující aritmetické operace nad celými čísly s predikátovým symbolem E interpretovaným jako množina sudých čísel:
 - $D_{I_5} = \mathbb{Z}$,
 - $I_5(+_{/2}) = +_{\mathbb{Z}}$ (tj. sčítání celých čísel),
 - $I_5(\cdot_{/2}) = \cdot_{\mathbb{Z}}$ (tj. násobení celých čísel),
 - $I_5(-_{/1}) = \{x \mapsto (0 -_{\mathbb{Z}} x) \mid x \in \mathbb{Z}\}$ (tj. opačné číslo) a
 - $I_5(E_{/1}) = \{\dots, -4, -2, 0, 2, 4, \dots\}$ (tj. sudá čísla).
6. Realizace I_6 modelující svaz na množině $\{a, b, c, d\}$ daný následujícím Hasseovým diagramem



kde

- $D_{I_6} = \{a, b, c, d\}$,
- interpretace symbolů $+/_2, \cdot/_2, -/_1$ jsou dány těmito tabulkami:

$I_6(+)$	a	b	c	d	$I_6(\cdot)$	a	b	c	d	$I_6(-)$	a	d
a	a	a	a	a	a	a	b	c	d	a	a	d
b	a	b	a	b	b	b	b	d	d	b	b	c
c	a	a	c	c	c	c	d	c	d	c	c	b
d	a	b	c	d	d	d	d	d	d	d	d	a

všimněte si, že $+$ je interpretováno jako supremum, $\cdot$ je interpretováno jako infimum a unární $-$ je interpretováno jako komplement;

- $I_6(E) = \{d\}$ (tj. infimum svazu).

2.2.1 Sémantika formule v realizaci

Sémantika formule je dána realizací, tedy formule samotná bez realizace nemá sémantiku; to znamená, že obecně nemůžeme říct, jestli formule platí nebo neplatí. Když realizaci I máme, můžeme pravdivostní hodnotu formule v realizaci I zjistit intuitivně tak, že do formule za proměnné, funkční a predikátové symboly dosadíme jejich realizace z I a formuli vyčíslíme. První věc kterou musíme spočítat je hodnota všech termů ve formuli pro danou realizaci.

Hodnota termu v realizaci

V pevně dané realizaci I s doménou D_I má term t konkrétní hodnotu: jeden prvek z domény D_I . Pro hodnotu termu t v realizaci I budeme používat notaci $I(t)$. Proměnné $x \in \mathbb{X}$ a konstantní funkční symboly $c/_0 \in \text{funcs}$ mají svou hodnotu přímo dány v realizaci jako $I(x)$ a $I(c)$. Hodnota komplexnějšího termu $t = f(t_1, \dots, t_n)$ pro n -ární funkční symbol f se definuje induktivně jako

$$I(f(t_1, \dots, t_n)) \stackrel{\text{def}}{=} f_I(I(t_1), \dots, I(t_n)) \quad \text{pro} \quad f_I = I(f).$$

Neformálně řečeno, vezme se funkce f_I která interpretuje symbol f v realizaci I a do té se dosadí hodnoty termů $t_1, \dots, t_n$; výsledkem bude opět nějaká hodnota z domény D_I .

Příklad 2.6

Spočítejme hodnotu následujících termů v různých realizacích z Příkladu 2.5.

1. $(x + y) + (z + x)$

(a) Uvažujme interpretaci I_1 z Příkladu 2.5 rozšířenou o interpretaci proměnných x, y, z takto: $I_1(x) = 3$, $I_1(y) = 42$, $I_1(z) = 17$. Hodnotu $I_1((x + y) + (z + x))$ pak můžeme spočítat následujícím způsobem:

$$\begin{aligned} I_1((x + y) + (z + x)) &= I_1(x + y) +_{\mathbb{N}} I_1(z + x) \\ &= (I_1(x) +_{\mathbb{N}} I_1(y)) +_{\mathbb{N}} (I_1(z) +_{\mathbb{N}} I_1(x)) \\ &= (3 +_{\mathbb{N}} 42) +_{\mathbb{N}} (17 +_{\mathbb{N}} 3) \\ &= 45 +_{\mathbb{N}} 20 \\ &= 65 \end{aligned}$$

(b) Uvažujme interpretaci I_2 rozšířenou o interpretaci proměnných x, y, z takto: $I_2(x) = [1, 02; 3, 57; 2, 62]$, $I_2(y) = [0; -3, 12; -4, 2]$, $I_2(z) = [-1, 123; 0, 35; 2, 56]$. Hodnotu termu $I_2((x + y) + (z + x))$ můžeme spočítat postupně následujícím způsobem:

$$\begin{aligned} I_2((x + y) + (z + x)) &= I_2(x + y) +_{\mathbb{R}^3} I_2(z + x) \\ &= (I_2(x) +_{\mathbb{R}^3} I_2(y)) +_{\mathbb{R}^3} (I_2(z) +_{\mathbb{R}^3} I_2(x)) \\ &= ([1, 02; 3, 57; 2, 62] +_{\mathbb{R}^3} [0; -3, 12; -4, 2]) +_{\mathbb{R}^3} (I_2(z) +_{\mathbb{R}^3} I_2(x)) \\ &= [1, 02; 0, 45; -1, 58] +_{\mathbb{R}^3} (I_2(z) +_{\mathbb{R}^3} I_2(x)) \\ &= [1, 02; 0, 45; -1, 58] +_{\mathbb{R}^3} ([-1, 123; 0, 35; 2, 56] +_{\mathbb{R}^3} [1, 02; 3, 57; 2, 62]) \\ &= [1, 02; 0, 45; -1, 58] +_{\mathbb{R}^3} [-0, 103; 3, 92; 5, 18] \\ &= [0, 917; 4, 37; 3, 6] \end{aligned}$$

(c) Uvažujme interpretaci I_3 rozšířenou o interpretaci proměnných x, y, z takto: $I_3(x) = 0$, $I_3(y) = 1$, $I_3(z) = 0$. Hodnota termu $I_3((x + y) + (z + x))$ bude

$$\begin{aligned} I_3((x + y) + (z + x)) &= I_3(x + y) +_{I_3} I_3(z + x) \\ &= (I_3(x) +_{I_3} I_3(y)) +_{I_3} (I_3(z) +_{I_3} I_3(x)) \\ &= (0 +_{I_3} 1) +_{I_3} (0 +_{I_3} 0) \\ &= 1 +_{I_3} 0 \\ &= 1 \end{aligned}$$

(d) Uvažujme interpretaci I_4 rozšířenou o interpretaci proměnných x, y, z takto: $I_4(x) = \text{"Hello"}$, $I_4(y) = \text{" world. "}$, $I_4(z) =$

"Just say ". Hodnota termu $I_4((x + y) + (z + x))$ bude (symbol „.“ značí konkatenci řetězců)

$$\begin{aligned}
 I_4((x + y) + (z + x)) &= I_4(x + y) \cdot I_4(z + x) \\
 &= (I_4(x) \cdot I_4(y)) \cdot (I_4(z) \cdot I_4(x)) \\
 &= ("Hello" \cdot " world. ") \cdot ("Just say " \cdot "Hello") \\
 &= "Hello world. " \cdot "Just say Hello" \\
 &= "Hello world. Just say Hello"
 \end{aligned}$$

2. $(x + y) \cdot (-z)$

(a) Uvažujme interpretaci I_5 z Příkladu 2.5 rozšířenou o interpretaci proměnných x, y, z takto: $I_5(x) = -17$, $I_5(y) = 42$, $I_5(z) = 3$. Hodnotu $I_5((x + y) \cdot (-z))$ pak můžeme spočítat následujícím způsobem:

$$\begin{aligned}
 I_5((x + y) \cdot (-z)) &= I_5(x + y) \cdot_{\mathbb{Z}} I_5(-z) \\
 &= (I_5(x) +_{\mathbb{Z}} I_5(y)) \cdot_{\mathbb{Z}} (0 -_{\mathbb{Z}} I_5(z)) \\
 &= (-17 +_{\mathbb{Z}} 42) \cdot_{\mathbb{Z}} (0 -_{\mathbb{Z}} 3) \\
 &= 25 \cdot_{\mathbb{Z}} -3 \\
 &= -75
 \end{aligned}$$

(b) Uvažujme interpretaci I_6 rozšířenou o interpretaci proměnných x, y, z takto: $I_6(x) = b$, $I_6(y) = d$, $I_6(z) = b$. Hodnota $I_6((x + y) \cdot (-z))$ bude následující ($\sqcap$ značí infimum, $\sqcup$ značí supremum a $\bar{w}$ značí komplement prvku w):

$$\begin{aligned}
 I_6((x + y) \cdot (-z)) &= I_6(x + y) \sqcap I_6(-z) \\
 &= (I_6(x) \sqcup I_6(y)) \sqcap \overline{I_6(z)} \\
 &= (b \sqcup d) \sqcap \bar{b} \\
 &= b \sqcap c \\
 &= d
 \end{aligned}$$

Pravdivostní hodnota formule v realizaci

Konečně se dostáváme k vyhodnocení pravdivostní hodnoty formule v realizaci. *Platnost* formule φ v realizaci I , značeno jako $I \models \varphi$ („ φ platí v I “ nebo „ I je *model* φ “), je definována induktivně následujícím způsobem:

1. Je-li φ atomická formule $p(t_1, \dots, t_m)$, pro m -ární predikátový symbol $p \in \mathcal{P}$, potom

$$I \models p(t_1, \dots, t_m) \quad \text{právě když} \quad (I(t_1), \dots, I(t_m)) \in p_I,$$

kde $p_I = I(p)$ je relace interpretující symbol p/m v I (pokud je symbol p unární, pak je p_I množina).

2. Je-li φ atomická formule $t_1 = t_2$ potom

$$I \models t_1 = t_2 \quad \text{právě když } I(t_1) \text{ a } I(t_2) \text{ jsou identické prvky.}$$

3. Pravdivostní hodnota pro výrokové spojky je definována očekávaným způsobem takto:

$$\begin{aligned} I \models \neg\varphi & \quad \text{právě když } I \not\models \varphi, \\ I \models \varphi \wedge \psi & \quad \text{právě když } I \models \varphi \text{ a } I \models \psi, \\ I \models \varphi \vee \psi & \quad \text{právě když } I \models \varphi \text{ nebo } I \models \psi, \\ I \models \varphi \rightarrow \psi & \quad \text{právě když pokud } I \models \varphi, \text{ pak } I \models \psi \text{ a} \\ I \models \varphi \leftrightarrow \psi & \quad \text{právě když buď } I \models \varphi \text{ a } I \models \psi, \text{ nebo } I \not\models \varphi \text{ a } I \not\models \psi, \end{aligned}$$

kde $I \not\models \varphi$ pokud neplatí $I \models \varphi$.

4. Pravdivostní hodnota existenčně a univerzálně kvantifikovaných formulí v realizaci I je definována tak, že

$$\begin{aligned} I \models \exists x\varphi & \quad \text{pokud existuje realizace } I', \text{ která rozšiřuje realizaci } I \\ & \quad \text{o ohodnocení proměnné } x \text{ na nějakou hodnotu z } D_I \\ & \quad \text{takové, že } I' \models \varphi \text{ a} \\ I \models \forall x\varphi & \quad \text{pokud pro libovolnou realizaci } I', \text{ která rozšiřuje reali-} \\ & \quad \text{zaci } I \text{ o ohodnocení proměnné } x \text{ na nějakou hodnotu} \\ & \quad \text{z } D_I \text{ platí, že } I' \models \varphi. \end{aligned}$$

Příklad 2.7

Uvažujme jazyk L predikátové logiky se signaturou $\langle \mathcal{F} = \{+, -, \cdot\}, \mathcal{P} = \{Z\} \rangle$ a jeho realizaci I_L s doménou $D_{I_L} = \{a, b, c\}$ a interpretací symbolů

$I_L(+)$	a	b	c		$I_L(-)$
a	a	b	c	a	a
b	b	c	a	b	c
c	c	a	b	c	b

a $I_L(Z) = \{a\}$.

Zkusíme určit, zda v realizaci I_L platí formule

$$\varphi: \forall x \forall y (Z(x) \rightarrow x + y = y).$$

Pro určení platnosti formule si můžeme na pomoc vzít tabulku obdobnou pravdivostní tabulce ve výrokové logice, která pro termy obsahuje jejich hodnoty z domény D_{I_L} a pro formule jejich pravdivostní hodnoty (tato tabulka lze vytvořit jen u realizací, jejichž doména je konečná):

x	y	$Z(x)$	$x + y$	$x + y = y$	$Z(x) \rightarrow x + y = y$	$\forall y(Z(x) \rightarrow x + y = y)$	$\forall x \forall y(Z(x) \rightarrow x + y = y)$
a	a	1	a	1	1	1	1
a	b	1	b	1	1		
a	c	1	c	1	1		
b	a	0	b	0	1	1	
b	b	0	c	0	1		
b	c	0	a	0	1		
c	a	0	c	0	1	1	
c	b	0	a	0	1		
c	c	0	b	0	1		

Podotkněme, že platnost formule v realizaci nejde vždy určit, viz následující poznámku.⁴

⁴ Jde o tzv. *nerozhodnutelný* problém, tedy o problém, pro jehož řešení neexistuje algoritmus. Více se o nerozhodnutelnosti dozvíte v předmětu Teoretická informatika (TIN).

Poznámka 2.2

Uvažme následující dvě formule v neformální syntaxi predikátové logiky:

$$\varphi_1: \forall x(x > 0 \rightarrow \exists n(f_1^n(x) = 1))$$

$$\varphi_2: \forall x(x > 0 \rightarrow \exists n(f_2^n(x) = 1))$$

kde f^n značí n -násobnou aplikaci funkce f , tj.

$$f^n(x) = \overbrace{(f \circ \dots \circ f)}^n(x)$$

a funkce f_1, f_2 jsou definovány následujícím způsobem:

$$f_1(x) = \begin{cases} \frac{x}{2} & \text{když } x \text{ je sudé} \\ x + 1 & \text{když } x \text{ je liché} \end{cases}$$

$$f_2(x) = \begin{cases} \frac{x}{2} & \text{když } x \text{ je sudé} \\ 3x + 1 & \text{když } x \text{ je liché} \end{cases}$$

Nechť $I_{\mathbb{N}}$ je standardní realizace elementární aritmetiky nad přirozenými čísly. Potom lze celkem jednoduše dokázat, že $I_{\mathbb{N}} \models \varphi_1$ (můžete si zkusit za domácí úkol). Platí i $I_{\mathbb{N}} \models \varphi_2$? Toto je od roku 1937 otevřený problém matematiky známý pod názvem *Collatzova domněnka* [8], který se dosud nikomu nepodařilo vyřešit. Může jít o vhodné téma bakalářské práce.

2.3 Terminologie

Formule φ v jazyce L je *splnitelná*, pokud má nějaký model, tedy pokud existuje nějaká realizace I jazyka L taková, že $I \models \varphi$.⁵

⁵ Splnitelnost hraje klíčovou roli mimo jiné v *logickém programování*, kde je program zadán jako množina formulí a interpret programovacího jazyka (např. Prologu) se snaží najít model této množiny formulí.

Příklad 2.8

Formule $\varphi: \forall x \forall y (x < y \rightarrow \exists z (x < z \wedge z < y))$ je splnitelná. Její modely jsou například následující realizace:

1. Realizace I_1 s doménou $D_{I_1} = \mathbb{Q}$ a standardní interpretací predikátového symbolu $<_{/2}$ (jako ostrá nerovnost) nad racionálními čísly.
2. Realizace I_2 s doménou $D_{I_2} = \mathbb{R}^3$ a následující interpretací symbolu $<_{/2}$:

$$[x_1, y_1, z_1] <_{I_2} [x_2, y_2, z_2] \stackrel{\text{def}}{\iff} \sqrt{x_1^2 + y_1^2 + z_1^2} < \sqrt{x_2^2 + y_2^2 + z_2^2}$$

3. Realizace I_3 s doménou $D_{I_3} = \mathbb{Z}$ a následující interpretací symbolu $<_{/2}$:

$$x <_{I_3} y \stackrel{\text{def}}{\iff} x <_{\mathbb{Z}} y \wedge x \text{ je sudé} \wedge y \text{ je sudé}.$$

4. Realizace I_4 s doménou $D_{I_4} = \mathbb{N}$ a prázdnou interpretací symbolu $p_{/2}$, tj. $I_4(p) = \emptyset$ (formule platí, protože antecedent v implikaci nebude nikdy platit, tedy celá implikace bude triviálně platit).
5. Realizace I_5 s doménou $D_{I_5} = \mathbb{N}$ a $I_5(<) = \mathbb{N} \times \mathbb{N}$ (tj. všechny dvojice čísel jsou v relaci).
6. Realizace I_6 s doménou $D_{I_6} = \{a\}$ a $I_6(<) = \emptyset$. Toto je model formule s nejmenší velikostí.

Naopak, modely formule nejsou např. tyto realizace:

7. Realizace I_7 s doménou $D_{I_7} = \mathbb{N}$ a standardní interpretací predikátového symbolu $<_{/2}$ nad přirozenými čísly.
8. Realizace I_8 s doménou $D_{I_8} = \{a, b\}$ a $I_8(<) = \{(a, b)\}$. Tato realizace je nejmenší realizace, která není modelem formule.

Formule φ je (*logicky*) *platná* pokud platí v libovolné realizaci. Jinými slovy, pro všechny realizace daného jazyka I platí $I \models \varphi$. Platnost φ značíme podobně jako ve výrokové logice: $\models \varphi$.

Příklad 2.9

Uvažujme jazyk se signaturou $\langle \mathcal{F} = \{1_{/0}, 2_{/0}, +_{/2}\}, \mathcal{P} = \emptyset \rangle$. Je formule

$$\varphi: 1 + 1 = 2$$

logicky platná?

Formule φ logicky platná není. Jaktože formule není logicky platná? Aby byla formule logicky platná, musí platit v *každé* realizaci daného jazyka. Podívejme se ale např. na realizaci I takovou, že

- $D_I = \{a, b\}$,
- $I(1) = a$,
- $I(2) = b$,
- $I(+)=\{(a,a)\mapsto a,\dots\}$,

V realizaci I je hodnota termu „ $1+1$ “ rovna $I(1+1)=a$ a hodnota termu „ 2 “ je rovna $I(2)=b$. Tyto hodnoty nejsou identické a tudíž I není modelem φ , a proto φ ani není logicky platná formule.

Dvě formule φ a ψ jsou (*logicky*) *ekvivalentní*, zapisováno $\varphi \Leftrightarrow \psi$, pokud pro všechny realizace I platí, že I je modelem φ právě tehdy, když I je modelem ψ .

Formule ψ je *logickým důsledkem* formule φ , zapisováno $\varphi \Rightarrow \psi$, tehdy, když pro každou realizaci I platí, že je-li I modelem φ , pak je I rovněž modelem ψ .

Poznámka 2.3

Výše uvedené vlastnosti (splnitelnost, platnost, ekvivalence, důsledek) jsou všechny obecně nerozhodnutelné (tedy, neexistuje obecný algoritmus, který by pro libovolnou formuli predikátové logiky uměl určit, zda platí nebo ne). Nerozhodnutelnost problému platnosti formule (tzv. *Entscheidungsproblem* [7]) dokázali nezávisle na sobě Alonzo Church pomocí výpočetního modelu *λ -kalkulu* v roce 1935 [2] a Alan Turing pomocí výpočetního modelu *Turingova stroje* v roce 1936 [5]. O obou těchto výpočetních modelech se dozvíte ve svém studiu více: *λ -kalkul* je formální základ *funkcionálních programovacích jazyků* (např. Haskell, OCaml, Scheme) a Turingův stroj je formální základ *imperativních programovacích jazyků*.

2.4 Algebraické úpravy formulí

Podobně jako ve výrokové logice, při práci s formulemi predikátové logiky budeme chtít provádět úpravy formulí zachovávající ekvivalenci. Při těchto úpravách budeme opět vycházet z logicky ekvivalentních formulí. Mimo logických ekvivalencí z výrokové logiky můžeme ještě využít ekvivalence z následující věty.

Věta 2.1. *Pro formule predikátové logiky φ a ψ platí následující ekviva-*

lence:

$$\begin{aligned}
\forall x\varphi &\Leftrightarrow \neg\exists x(\neg\varphi) & \exists x\varphi &\Leftrightarrow \neg\forall x(\neg\varphi) \\
\forall x\varphi &\Leftrightarrow \varphi & \exists x\varphi &\Leftrightarrow \varphi & \text{pokud } x \notin \text{FREE}[\varphi] \\
\forall x(\varphi \vee \psi) &\Leftrightarrow (\forall x\varphi) \vee \psi & \exists x(\varphi \wedge \psi) &\Leftrightarrow (\exists x\varphi) \wedge \psi & \text{pokud } x \notin \text{FREE}[\psi] \\
\forall x(\varphi \rightarrow \psi) &\Leftrightarrow (\exists x\varphi) \rightarrow \psi & \exists x(\varphi \rightarrow \psi) &\Leftrightarrow (\forall x\varphi) \rightarrow \psi & \text{pokud } x \notin \text{FREE}[\psi] \\
\forall x(\varphi \rightarrow \psi) &\Leftrightarrow \varphi \rightarrow (\forall x\psi) & \exists x(\varphi \rightarrow \psi) &\Leftrightarrow \varphi \rightarrow (\exists x\psi) & \text{pokud } x \notin \text{FREE}[\varphi] \\
\\
(\forall x(\varphi(x))) \wedge (\forall y(\psi(y))) &\Leftrightarrow \forall x(\varphi(x) \wedge \psi(x)) & \text{pokud } x \notin \text{FREE}[\psi] \\
(\exists x(\varphi(x))) \vee (\exists y(\psi(y))) &\Leftrightarrow \exists x(\varphi(x) \vee \psi(x)) & \text{pokud } x \notin \text{FREE}[\psi]
\end{aligned}$$

Příklad 2.10

Ukážeme si, jak převést formuli

$$\neg\forall x\exists y\left(\neg((p(x) \wedge r(x)) \rightarrow r(y))\right)$$

do tvaru, ve kterém jsou kvantifikátory co nejhlouběji:

$$\begin{aligned}
&\neg\forall x\exists y\left(\neg((p(x) \wedge r(x)) \rightarrow r(y))\right) \\
&\Leftrightarrow \neg\forall x\left(\neg\forall y((p(x) \wedge r(x)) \rightarrow r(y))\right) \\
&\Leftrightarrow \neg\neg\exists x\forall y((p(x) \wedge r(x)) \rightarrow r(y)) \\
&\Leftrightarrow \exists x\forall y((p(x) \wedge r(x)) \rightarrow r(y)) \\
&\Leftrightarrow \exists x((p(x) \wedge r(x)) \rightarrow \forall y(r(y))) \\
&\Leftrightarrow (\forall x(p(x) \wedge r(x)) \rightarrow \forall y(r(y))) \\
&\Leftrightarrow ((\forall x(p(x))) \wedge (\forall x(r(x)))) \rightarrow \forall y(r(y))
\end{aligned}$$

Výsledná ekvivalentní formule s kvantifikátory co nejhlouběji je tedy

$$((\forall x(p(x))) \wedge (\forall x(r(x)))) \rightarrow \forall y(r(y)).$$

2.5 Prenexní normální forma

Základní normální forma v predikátové logice je tzv. *prenexní normální forma*. Tato forma slouží jako základ pro mnoho další úprav formulí, jako je například *Skolemizace* (viz další sekce).

Formule φ je v *prenexní normální formě* (PNF), pokud začíná prefixem kvantifikátorů, za nímž následuje formule bez kvantifikátorů, tj. má následující tvar:

$$\varphi: Q_1x_1Q_2x_2\ldots Q_kx_k(\psi(x_1,\ldots,x_k)),$$

kde $Q_1, \dots, Q_k \in \{\exists, \forall\}$, $x_1, \dots, x_k \in \mathbb{X}$ a ψ je formule bez kvantifikátorů.

Pro převod formule do PNF můžeme použít algebraické úpravy z vý-
rokové logiky a ze Sekce 2.4.

Příklad 2.11

Upravíme formuli

$$\forall n(n > 2 \rightarrow \neg \exists x \exists y \exists z (x^n + y^n = z^n))$$

do PNF:

$$\begin{aligned} & \forall n(n > 2 \rightarrow \neg \exists x \exists y \exists z (x^n + y^n = z^n)) \\ & \Leftrightarrow \forall n(n > 2 \rightarrow \forall x \neg \exists y \exists z (x^n + y^n = z^n)) \\ & \Leftrightarrow \forall n(n > 2 \rightarrow \forall x \forall y \neg \exists z (x^n + y^n = z^n)) \\ & \Leftrightarrow \forall n(n > 2 \rightarrow \forall x \forall y \forall z (\neg (x^n + y^n = z^n))) \\ & \Leftrightarrow \forall n \forall x (n > 2 \rightarrow \forall y \forall z (\neg (x^n + y^n = z^n))) \\ & \Leftrightarrow \forall n \forall x \forall y (n > 2 \rightarrow \forall z (\neg (x^n + y^n = z^n))) \\ & \Leftrightarrow \forall n \forall x \forall y \forall z (n > 2 \rightarrow \neg (x^n + y^n = z^n)) \end{aligned}$$

Výsledná formule v PNF je

$$\forall n \forall x \forall y \forall z (n > 2 \rightarrow \neg (x^n + y^n = z^n)).$$

Příklad 2.12

Upravíme formuli

$$\forall y \left(\exists x (P(x, y) \rightarrow Q(y, z)) \right) \wedge \neg \exists z \left(\forall x (R(x, y) \vee Q(x, y)) \right)$$

do PNF:

$$\begin{aligned} & \forall y \left(\exists x (P(x, y) \rightarrow Q(y, z)) \right) \wedge \neg \exists z \left(\forall x (R(x, y) \vee Q(x, y)) \right) \\ & \Leftrightarrow \forall y \left(\exists x (P(x, y) \rightarrow Q(y, z)) \right) \wedge \neg \forall x (R(x, y) \vee Q(x, y)) \\ & \Leftrightarrow \forall y \left(\exists x (P(x, y) \rightarrow Q(y, z)) \right) \wedge \neg \forall u (R(u, y) \vee Q(u, y)) \\ & \Leftrightarrow \forall y \left(\exists x (P(x, y) \rightarrow Q(y, z)) \right) \wedge \exists u \left(\neg (R(u, y) \vee Q(u, y)) \right) \\ & \Leftrightarrow \forall y \left(\left(\exists x (P(x, y) \rightarrow Q(y, z)) \right) \wedge \exists u \left(\neg (R(u, y) \vee Q(u, y)) \right) \right) \\ & \Leftrightarrow \forall y \exists u \left(\left(\exists x (P(x, y) \rightarrow Q(y, z)) \right) \wedge \neg (R(u, y) \vee Q(u, y)) \right) \\ & \Leftrightarrow \forall y \exists u \left(\forall x (P(x, y) \rightarrow Q(y, z)) \wedge \neg (R(u, y) \vee Q(u, y)) \right) \\ & \Leftrightarrow \forall y \exists u \forall x \left((P(x, y) \rightarrow Q(y, z)) \wedge \neg (R(u, y) \vee Q(u, y)) \right) \end{aligned}$$

Výsledná formule v PNF je

$$\forall y \exists u \forall x \left((P(x, y) \rightarrow Q(y, z)) \wedge \neg (R(u, y) \vee Q(u, y)) \right).$$

2.6 Skolemova normální forma

Další normální forma v predikátové logice je tzv. *Skolemova normální forma*. Tato normální forma je používána hlavně např. při automatickém dokazování formulí predikátové logiky pomocí tzv. *automatizovaných theorem proverů* (angl. *automated theorem provers*) založených na technikách rezoluce, *superpozice*, *paramodulace*, atd.

Formule je ve *Skolemově normální formě* (SNF), pokud je v prenexní normální formě a neobsahuje existenční kvantifikátory $\exists$. Zde je potřeba uvědomit si, že obecně neplatí, že by ke každé formuli predikátové logiky existovala formule v SNF, která by byla s původní formulí *logicky ekvivalentní*. Platí ale, že ke každé formuli φ existuje formule φ' taková, že φ' je *splnitelná* právě když je *splnitelná* formule φ (používá se termín *ekvisplnitelnost*). Procesu získání ekvisplnitelné formule v SNF se říká *Skolemizace*.

Základní myšlenka Skolemizace je jednoduchá. Předpokládejme, že máme následující formuli v PNF:

$$\varphi: \forall x_1 \forall x_2 \dots \forall x_k \exists y (\psi(x_1, \dots, x_k, y)).$$

Tato formule bude splnitelná právě tehdy, když pro každou k -tici hodnot z domény potenciální realizace I , $(v_1, \dots, v_k) \in D_I^k$, platí, že existuje hodnota $u \in D_I$ taková, že formule $\psi(x_1, \dots, x_k, y)$ je splnitelná s ohodnocením proměnných $I(x_1) = v_1, \dots, I(x_k) = v_k$ a $I(y) = u$. Snadno lze nahlédnout, že existence daného u pro každou k -tici $(v_1, \dots, v_k)$ je ekvivalentní existenci k -ární funkce f_y , která každé k -tici hodnot přiřazuje hodnotu u tak, aby byla formule splněna. Můžeme tedy ve formuli φ odstranit kvantifikátor $\exists y$ a nahradit všechny volné výskyty proměnné y ve formuli ψ za term $f_y(x_1, \dots, x_k)$. Obdrželi bychom tedy formuli

$$\varphi': \forall x_1 \forall x_2 \dots \forall x_k (\psi(x_1, \dots, x_k, f_y(x_1, \dots, x_k))).$$

Povšimněte si, že jsme do jazyka zavedli nový funkční symbol f_y ; tomuto funkčnímu symbolu se říká *Skolemova funkce*. Obsahuje-li formule více existenčních kvantifikátorů, Skolemizace postupně odstraňuje kvantifikátory od nejvnějššího po nejvnitřnější.

Příklad 2.13

Převédeme následující formuli na ekvisplnitelnou formuli ve Skolemově normální formě:

$$\forall y \exists u \forall x \left((P(x, y) \rightarrow Q(y, z)) \wedge \neg (R(u, y) \vee Q(u, y)) \right).$$

Formule již je v PNF, můžeme tedy rovnou pokračovat Skolemizací:

$$\begin{aligned} & \forall y \exists u \forall x \left((P(x, y) \rightarrow Q(y, z)) \wedge \neg (R(u, y) \vee Q(u, y)) \right) \\ & \Leftrightarrow \forall y \forall x \left((P(x, y) \rightarrow Q(y, z)) \wedge \neg (R(f_u(y), y) \vee Q(f_u(y), y)) \right) \end{aligned}$$

Příklad 2.14

Převědeme následující formuli na ekvivalentní formuli ve Skolemově normální formě:

$$\exists v \forall x \exists y \forall u \exists z (v + (x + y) = u + z).$$

Formule opět již je v PNF, můžeme tedy rovnou pokračovat Skolemizací:

$$\begin{aligned} & \exists v \forall x \exists y \forall u \exists z (v + (x + y) = u + z) \\ & \Leftrightarrow \forall x \exists y \forall u \exists z (f_v + (x + y) = u + z) \\ & \Leftrightarrow \forall x \forall u \exists z (f_v + (x + f_y(x)) = u + z) \\ & \Leftrightarrow \forall x \forall u (f_v + (x + f_y(x)) = u + f(x, u)) \end{aligned}$$

Literatura

- [1] J. W. Backus. 1959. The syntax and semantics of the proposed international algebraic language of the Zurich ACM-GAMM conference. In *Proceedings of the international conference on information processing*, UNESCO, 125–132.
- [2] Alonzo Church. 1936. An unsolvable problem of elementary number theory. *American Journal of Mathematics* 58, (1936), 345.
- [3] Herbert B. Enderton. 2009. Second-order and Higher-order Logic. In *The Stanford encyclopedia of philosophy* (Winter 2018), Edward N. Zalta (ed.). <https://plato.stanford.edu/archives/win2018/entries/logic-higher-order/>; Metaphysics Research Lab, Stanford University.
- [4] Peter Naur. 1961. A course of Algol 60 programming. *ALGOL Bull.* Sup 9 (January 1961), 1–38.
- [5] A. M. Turing. 1937. On computable numbers, with an application to the Entscheidungsproblem. *Proceedings of the London Mathematical Society* s2-42, 1 (1937), 230–265.
- [6] Wikipedia contributors. 2021. Fermat’s last theorem — Wikipedia, the free encyclopedia. https://en.wikipedia.org/w/index.php?title=Fermat%27s_Last_Theorem.
- [7] Wikipedia contributors. 2021. Entscheidungsproblem — Wikipedia, the free encyclopedia. <https://en.wikipedia.org/w/index.php?title=Entscheidungsproblem>.
- [8] Wikipedia contributors. 2021. Collatz conjecture — Wikipedia, the free encyclopedia. https://en.wikipedia.org/w/index.php?title=Collatz_conjecture.